

BCJ



BCJ37 - Dossier projet

Formation : Titre professionnel Développeur Web

Apprenant : Alexandre Bourlier

Année : 2026



**Centre Européen
de Formation**

Assurez votre succès

1) INTRODUCTION	1
2) COMPETENCES DU REFERENTIEL COUVERTES PAR LE PROJET	1
I. CCP 1 — DEVELOPPER LA PARTIE FRONT-END D'UNE APPLICATION WEB OU WEB MOBILE SECURISEE	2
II. CCP 2 — DEVELOPPER LA PARTIE BACK-END D'UNE APPLICATION WEB OU WEB MOBILE SECURISEE	2
III. COMPETENCES TRANSVERSALES.....	2
3) CONTEXTE DU PROJET.....	3
4) EXPRESSION DES BESOINS.....	3
1. <i>Besoins fonctionnels :</i>	3
2. <i>Besoins non fonctionnels :</i>	3
3. <i>Limites du projet</i>	3
5) ETUDE DE L'EXISTANT.....	3
6) CONTRAINTES ET LIVRABLES ATTENDUS	4
7) ENVIRONNEMENT HUMAIN ET TECHNIQUE	4
1. <i>Environnement humain.....</i>	4
2. <i>Environnement technique de développement</i>	4
8) JUSTIFICATION DES CHOIX TECHNIQUES (REACT VS BLADE)	5
I. CONTEXTE INITIAL.....	6
II. LIMITES DE L'APPROCHE BLADE.....	6
III. CHOIX DE REACT.....	7
1. <i>Découplage frontend / backend</i>	7
2. <i>Architecture orientée composants</i>	7
3. <i>Navigation fluide (SPA).....</i>	7
4. <i>Gestion des données dynamique</i>	7
5. <i>Typage avec TypeScript.....</i>	7
IV. CONCLUSION.....	8
9) REALISATIONS FRONT-END (CCP1)	8
I. MAQUETTE DES INTERFACES UTILISATEURS (CP2).....	8
II. SCHEMA D'ENCHAINEMENT DES MAQUETTES (CP2).....	9
III. INTERFACES UTILISATEUR STATIQUES (CP3)	9
1. <i>Démarche.....</i>	9
2. <i>Structure sémantique et accessibilité.....</i>	9
3. <i>Responsivité et SEO</i>	10
4. <i>Partie dynamique des interfaces</i>	11
a) <i>Consommation de l'API REST</i>	11
b) <i>Chargement asynchrone et gestion des états.....</i>	12
c) <i>Interactivité, accessibilité et SEO dynamique.....</i>	12
d) <i>Point de sécurité assumé : contenu riche des articles.....</i>	13
IV. SYNTHESE	13
10) ACCESSIBILITE ET EXPERIENCE UTILISATEUR (UX).....	13
I. OBJECTIFS	13
II. ACCESSIBILITE	13
1. <i>Structure sémantique.....</i>	13
2. <i>Attributs ARIA.....</i>	14

3.	<i>Contenus alternatifs</i>	14
4.	<i>Formulaire accessible</i>	14
III.	EXPERIENCE UTILISATEUR (UX)	14
1.	<i>Navigation claire</i>	14
2.	<i>Organisation des contenus</i>	14
3.	<i>Feedback utilisateur</i>	15
4.	<i>Performance perçue</i>	15
5.	<i>Intégration de contenus externes</i>	15
11)	REALISATION BACK-END	15
I.	MISE EN PLACE DE LA BASE DE DONNEES RELATIONNELLE	15
1.	<i>Du modèle conceptuel au modèle physique</i>	15
2.	<i>Script de création de la base</i>	17
3.	<i>Règles appliquées</i>	17
II.	COMPOSANTS D'ACCES AUX DONNEES.....	18
1.	<i>Modèle et relation</i>	18
2.	<i>Validation des entrées et format de réponse</i>	18
3.	<i>Aspect « non relationnel »</i>	19
III.	COMPOSANTS METIER COTE SERVEUR	19
1.	<i>Un composant métier simple et testable</i>	19
2.	<i>Architecture métier avancée : le pipeline d'import des licenciés</i>	20
3.	<i>Fonction scraping / import des licenciés</i>	21
e)	Objectif du système d'import	21
f)	Positionnement dans l'architecture globale	22
g)	Fonctionnement global du pipeline	22
h)	Import batch et validation	23
i)	Organisation technique Laravel	24
j)	Gestion des erreurs et tests.....	24
k)	Cas concret d'utilisation	25
l)	Bilan.....	25
IV.	SYNTHESE	26
12)	SECURITE DE L'APPLICATION	26
I.	MESURES MISES EN PLACE	26
1.	<i>Authentification API et protection des routes sensibles</i>	26
2.	<i>CORS restrictif</i>	27
3.	<i>Rate limiting</i>	27
II.	INTERET DES MESURES	28
III.	CONCLUSION.....	28
13)	FONCTIONNALITE TESTEE	29
14)	VEILLE SUR LA SECURITE	31
I.	ANALYSE DES PRINCIPALES VULNERABILITES OWASP ET MESURES PREVENTIVES	31
II.	SYNTHESE DE LA VEILLE SECURITE	33
15)	DEPLOIEMENT	34
I.	MISE EN PRODUCTION – INTRODUCTION	34
II.	PREPARATION DE L'HEBERGEMENT	34
1.	<i>Création des sous-domaines</i>	34
2.	<i>Certificats SSL</i>	34

3.	<i>Arborescence des dossiers</i>	35
III.	PREPARATION DE LA BASE DE DONNEES	35
IV.	DEPLOIEMENT DU BACKEND	36
V.	DEPLOIEMENT DU FRONTEND (REACT).....	37
VI.	CONNEXION FRONTEND <-> API	37
1.	<i>Configuration de l'URL de l'API</i>	37
2.	<i>Fonctionnement du CORS</i>	38
3.	<i>Résolution du CORS en production</i>	38
VII.	SECURITE	38
VIII.	TESTS DE VALIDATION	39
1.	<i>Tests de l'API</i>	39
2.	<i>Tests frontend</i>	39
3.	<i>Test du formulaire de contact</i>	40
4.	<i>Vérifications finales</i>	40
IX.	BILAN DE LA MISE EN PRODUCTION	40
1.	<i>Difficultés rencontrées</i>	40
2.	<i>Solutions apportées</i>	41
X.	BUGS RENCONTRES ET RESOLUTIONS.....	41
16)	CONCLUSION	43
17)	ANNEXES	44
I.	ANNEXE 1 – ARCHITECTURE GLOBAL REACT / LARAVEL	44
II.	ANNEXE 2 – PIPELINE D'IMPORT CUESCORE	44
III.	ANNEXE 3 – ZONING – WIREFRAME ET MAQUETTE	45
IV.	ANNEXE 4 – SCRIPT DE CREATION DE LA BASE.....	46
V.	ANNEXE 5 – CODE COMPOSANTS METIER	51
1.	<i>Orchestrator du pipeline d'import des licenciés</i>	51
2.	<i>Composant métier de correspondance discipline / identifiant</i>	53
VI.	ANNEXE 6 – CODE DES COMPOSANTS D'ACCES AUX DONNEES	54
1.	<i>Modèle Eloquent Calendar (relations, filtres réutilisables, sécurité)</i>	54
2.	<i>Contrôleur d'API CalendarController (validation + réponse standardisée)</i>	55
VII.	DOCUMENTATION LARAVEL	56
VIII.	DOCUMENTATION REACT ET VITE	57
IX.	DOCUMENTATION OPENADMIN	57
X.	DOCUMENTATION MySQL.....	58
XI.	HEBERGEMENT ET DEPLOIEMENT	58
XII.	RESSOURCES COMPLEMENTAIRES.....	58

1) Introduction

Dans le cadre de la formation Développeur Web et Web Mobile du Centre Européen de Formation, ce dossier présente le projet de refonte et de modernisation du site web **BCJ37** (Billard club de Joué-lès-Tours).

L'objectif principal du projet était de faire évoluer une application web existante reposant sur Laravel et Blade vers une architecture moderne découplée, basée sur un frontend React et une API REST Laravel. Cette transformation visait à améliorer la maintenabilité du projet, l'expérience utilisateur, la gestion des données et la préparation à de futures évolutions fonctionnelles.

Le projet s'inscrit dans un contexte réel de besoins associatifs ; le club souhaitait disposer d'une plateforme plus moderne permettant :

- La consultation des actualités et informations du club ;
- L'affichage des classements et compétitions ;
- L'intégration de données externes provenant de CueScore ;
- L'automatisation de la gestion des licenciés ;
- Une meilleure compatibilité mobile ;
- Une amélioration globale des performances et de l'accessibilité.

La réalisation de ce projet a nécessité la mise en œuvre de nombreuses compétences techniques liées au développement web full stack :

- Conception d'API REST ;
- Développement frontend avec React et TypeScript ;
- Utilisation du framework Laravel ;
- Gestion d'une base de données MySQL ;
- Sécurisation des échanges ;
- Déploiement d'une application web en environnement distant ;
- Automatisation de traitements serveur ;
- Résolution de problématiques liées au CORS, au HTTPS et au routage SPA.

Ce dossier présente l'ensemble des étapes du projet, depuis l'analyse de l'existant jusqu'à la mise en production, en détaillant les choix techniques réalisés, l'architecture mise en place, les problématiques rencontrées ainsi que les solutions apportées.

2) Compétences du référentiel couvertes par le projet

Le projet BCJ37 met en œuvre les compétences du titre professionnel Développeur web et web mobile (TP-01280), réparties sur ses deux blocs de compétences (CCP). Le tableau ci-dessous indique, pour chaque compétence, où elle est démontrée dans ce dossier.

I. CCP 1 — Développer la partie front-end d'une application web ou web mobile sécurisée

Compétence	Démontrée dans
CP1 — Installer et configurer son environnement de travail	Contexte : Environnement humain et technique (Laragon, Git/GitHub, Composer, npm/Vite)
CP2 — Maquetter des interfaces utilisateur	Réalisations front-end : Maquette des interfaces + Schéma d'enchaînement des maquettes
CP3 — Réaliser des interfaces utilisateur statiques	Réalisations front-end : Interfaces utilisateur statiques (composants React, responsive, SEO)
CP4 — Développer la partie dynamique des interfaces	Réalisations front-end : Partie dynamique (consommation de l'API REST, asynchrone, gestion des états)

II. CCP 2 — Développer la partie back-end d'une application web ou web mobile sécurisée

Compétence	Démontrée dans
CP5 — Mettre en place une base de données relationnelle	Réalisation back-end : Mise en place de la base de données (MCD, MLD, script de création)
CP6 — Développer des composants d'accès aux données SQL et NoSQL	Réalisation back-end : Composants d'accès aux données (Eloquent, validation, JSON Cuescore)
CP7 — Développer des composants métier côté serveur	Réalisation back-end : Composants métier (services, pipeline d'import des licenciés)
CP8 — Documenter le déploiement d'une application dynamique	Déploiement (mise en production, procédure et scripts)

III. Compétences transversales

Compétence	Démontrée dans
Communiquer en français et en anglais	Rédaction du dossier ; compréhension de la documentation technique (Laravel, React) en anglais
Mettre en œuvre une démarche de résolution de problème	Bugs rencontrés et résolutions (diagnostic, tests, correction)
Apprendre en continu	Veille sur la sécurité (OWASP, CERT-FR) et montée en compétence React / architecture DDD

Note : Le projet ayant été réalisé à titre personnel et bénévole pendant la formation, l'ensemble des compétences a été mis en œuvre en autonomie, de la conception au déploiement.

3) Contexte du projet

Le billard club de Joué-Lès-Tours (BCJ37) est une association sportive d'Indre-et-Loire affiliée à la fédération française de Billard. Le club regroupe plusieurs disciplines, carambole, snooker et blackball et participe à des compétitions départementales, régionales, nationales et internationales.

Son site vitrine est la **vitrine publique** du club : il diffuse les actualités, les calendriers de compétitions par discipline, les classements des joueurs, les documents officiels (règlements, documents officiels, code sportifs, ...) et les informations pratiques (présentations du club, partenaires, contact).

J'ai réalisé ce projet à **titre personnel et bénévole**, en mettant à profit la formation pour appliquer concrètement les compétences acquises sur un cas réel et utile au club.

4) Expression des besoins

En l'absence de commanditaire formel, j'ai recueilli les besoins auprès des responsables du club et je les ai formalisés afin de définir les objectifs et les limites du projet.

1. Besoins fonctionnels :

- Consulter les **actualités** du club (articles avec image, contenu et archives par année) ;
- Afficher les **calendriers de compétitions** filtrables par discipline et par niveau ;
- Présenter les **classements** des joueurs, alimentées depuis la plateforme externe CueScore ;
- Mettre à disposition des **documents** téléchargeables par discipline ;
- Proposer les **pages institutionnelles** (club, mentions légales, contact avec formulaire) ;
- Permettre aux responsables de **gérer ces contenus** sans connaissances technique, via une interface d'administration.

2. Besoins non fonctionnels :

- Un site **rapide, responsive** (mobile, tablette, ordinateur) et **bien référencé** ;
- Une application **maintenable** et **évolutive** dans le temps ;
- Le **respect du RGPD**, adapté à un contexte associatif ;
- La **sécurité** des données et de l'administration.

3. Limites du projet

Le périmètre couvre le site public, son API et son administration. Il exclut la gestion fédérale des licences (assurée par la fédération) ; le projet se limite à **consommer** les données exposées par CueScore.

5) Etude de l'existant

L'ancien site reposait sur une architecture **Laravel monolithique** (vues Blade) devenue difficile à maintenir : logique métier dupliquée, **une table de calendrier par discipline et par niveau** (ex : *blackball_calendrier_national*, *snooker_calendrier_regional*, ...), pages lourdes, performances et référencement limités (score Lighthouse = 66). Ces constats ont directement motivé une **refonte complète** plutôt qu'une simple correction, et ont orienté les choix techniques du chapitre 7.

6) Contraintes et livrables attendus

Type	Détail
Contraintes techniques	Hébergement mutualisé associatif, base MySQL , budget nul (briques open source)
Contraintes de délai	Travail bénévole mené en parallèle de la formation
Contraintes légales	RGPD, mentions légales, accessibilité
Livrables	Site public React, API REST Laravel, administration des contenus, base de données, documentation

7) Environnement humain et technique

1. Environnement humain

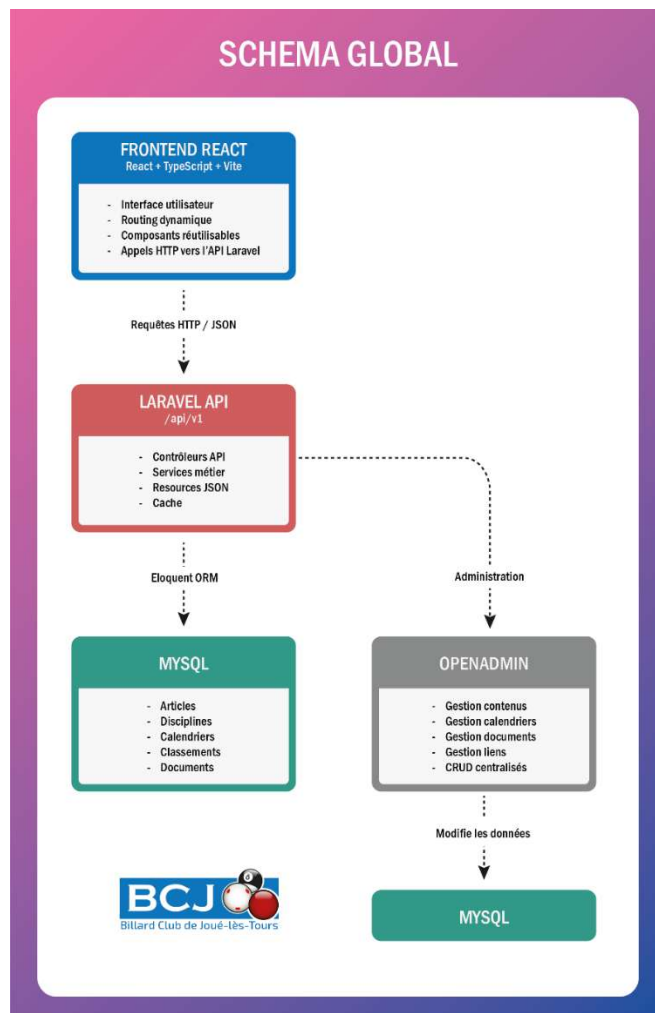
Projet mené en **autonomie** : j'ai assuré la conception le développement front-end et back-end, la base de données et le déploiement. Les responsables du club ont joué le rôle de **référents fonctionnels** (validation des besoins et des contenus).

2. Environnement technique de développement

- Poste sous **Windows** avec l'environnement local **Laragon** (Apache, MySQL, PHP 8.3) ;
- Editeur **VS Code** ;
- Gestion de versions **Git** et dépôt distant **GitHub** (branches de fonctionnalités, pull requests) ;
- Gestion des dépendances **Composer** (PHP) et **npm** (JavaScript), build front avec **Vite** ;
- Documentation d'API générée avec **Scribe**, audits avec **Lighthouse**, mesure d'audience avec **Matomo** auto-hébergé.

8) Justification des choix techniques (React vs Blade)

Les besoins (maintenabilité, performances, référencement, évolutivité) ont conduit à abandonner le monolithique Blade au profit d'une **architecture découplée « API-first »**, organisée en trois briques distinctes :



Choix	Alternatives écartées	Pourquoi ce choix
Laravel 10 (API-first) côté serveur	Symfony, monolithe Blade	Ecosystème mature, Eloquent ORM, validation et sécurité intégrées ; le mode API-first sépare nettement données et présentation
React 19 + TypeScript côté client	Vue, rester en Blade	Composants réutilisables, rendu dynamique fluide ; TypeScript fiabilise le code par le typage statique
MySQL (relationnel)	NoSQL	Données fortement structurées et reliées (disciplines, calendriers, événements, classements) -> le relationnel garantit

		l'intégrité par les clés étrangères.
OpenAdmin pour l'administration	Back-office sur mesure	Génération rapide de CRUD sécurisés, isolée du site public, prise en main simple par des bénévoles non techniciens
Laravel Sanctum	Sessions classiques	Authentification adaptée à une API et à l'administration
Matomo auto-hébergé	Google Analytics	Respect de la vie privée et conformité RGPD, pertinent pour une association.

Le bénéfice le plus visible de cette architecture concerne les **calendriers** : les multiples tables par discipline ont été remplacées par un modèle **unifié et normalisé** (calendars, calendar_events, calendar_event_links), détaillé au chapitre (à modifier plus tard). Cette refonte a réduit la duplication de code, simplifié l'administration et amélioré les performances (**Lighthouse** = 66 -> ~91, et **SEO, accessibilité et bonnes pratiques** portés à 100).

I. Contexte initial

Le projet était initialement basé sur une architecture classique Laravel utilisant **Blade + contrôleurs serveur + rendu côté serveur**.

Cette approche permet une mise en place rapide, mais présente certaines limites dans un contexte d'évolution du projet.

II. Limites de l'approche Blade

L'utilisation de Blade implique :

- Un couplage fort entre backend et frontend
- Une logique parfois mélangée entre :
 - o Traitement des données
 - o Affichage
- Des rechargements complets de page
- Une difficulté de réutilisation des composants UI
- Une évolution plus complexe vers des fonctionnalités dynamiques (SPA)

Dans le cadre du projet, ces limites devenaient problématiques notamment pour :

- La gestion des disciplines dynamiques
- L'affichage conditionnel des contenus
- L'intégration de sources externes (CueScore, PDF, Calendrier)
- La navigation fluide

III. Choix de React

Le passage à React s'inscrit dans une volonté de modernisation de l'application.

1. Découplage frontend / backend

Avec React :

- **Frontend -> React (client)**
- **Backend -> Laravel (API)**

Avantages :

- Séparation claire des responsabilités
- Meilleure maintenabilité
- Possibilité d'évolution indépendante

2. Architecture orientée composants

React permet une structuration basée sur des composants réutilisables :

- **PostsSection**
- **CalendarSection**
- **DocumentsSection**
- **RankingsSection**

Avantages :

- Factorisation du code
- Lisibilité
- Facilité de maintenance

3. Navigation fluide (SPA)

Grâce à React Router :

- Navigation sans rechargement de page
- Meilleure expérience utilisateur
- Gestion dynamique des routes :
 - o ***/disciplines/:discipline***
 - o ***/posts/:slug***
 - o ***/club/année/:period***

4. Gestion des données dynamique

Le frontend consomme l'API Laravel via des appels HTTP :

- Récupération des données à la demande
- Gestion des états :
 - o Loading
 - o Erreur
 - o Succès

Cela permet une interface plus réactive et adaptée aux usages modernes.

5. Typage avec TypeScript

L'utilisation de TypeScript permet :

- Une meilleure fiabilité du code
- La détection d'erreurs en amont
- Une documentation implicite des données API

IV. Conclusion

Le passage de Blade à React permet une application plus moderne, modulaire et évolutive.
Ce choix est particulièrement adapté à

- Un projet amené à évoluer
- Une gestion de contenu dynamique
- Une amélioration de l'expérience utilisateur

9) Réalisations front-end (CCP1)

Le front-end est une application **React 19 + TypeScript**, construite avec **Vite** et routée par **React Router**. Elle consomme l'API REST Laravel et n'utilise plus aucune vue Blade.

I. Maquette des interfaces utilisateurs (CP2)

A partir de l'expression des besoins (Chapitre 3), j'ai réalisé sous **Figma** la maquette de l'écran le plus représentatif du site, la **page des classements Blackball**, en respectant la charte graphique du club (couleurs, logo) et en la déclinant en **version web (ordinateur)** et **version web mobile**.

Pourquoi cet écran ? la page des classements Blackball est la plus **riche en données** (tableau de classement avec rang, joueur, points) et la **plus complexe à mettre en page** (densité d'information, lisibilité, adaptation mobile d'un tableau). C'est donc la maquette la plus **démonstrative** des contraintes d'interface du projet. Les autres écrans ont été développés directement à partir de la charte graphique et du schéma d'enchaînement, puis sont présentés via leurs **captures d'écran** réelles.

Ecran maqueté	Rôle	Web	Mobile
Classements Blackball	Affichage du classement (rang, joueur, points)		

Prise en compte de l'accessibilité et de l'expérience utilisateur dès la maquette : hiérarchie visuelle claire, contrastes suffisants, **lisibilité du tableau** et passage à un affichage adapté sur mobile (zones tactiles dimensionnées), navigation simplifiée. **Sécurité** : la gestion des contenus (saisie/mise à jour des classements) relève de l'administration, distincte du site public et non accessible depuis celui-ci.

II. Schéma d'enchaînement des maquettes (CP2)

L'enchaînement des écrans (navigation entre l'accueil, les pages disciplines, le détails d'un article et les pages institutionnelles) est formalisée par un schéma.

[Insertion schéma enchaînements]

Ce schéma traduit directement le **routing** mis en place côté React :

/	→ Accueil
/disciplines/:discipline	→ Page discipline (articles, calendrier, classements, documents)
/posts/:slug	→ Détail d'un article
/club, /contact	→ Pages institutionnelles

III. Interfaces utilisateur statiques (CP3)

1. Démarche

Les interfaces sont découpées en **composants réutilisables** (principe DRY) afin de garantir la cohérence visuelle et de simplifier la maintenance. Le code est écrit en **TSX** (JSX typé) et la mise en forme repose sur des classes CSS dédiées.

Exemple de composant de présentation réutilisable (la carte d'article) :

```
// frontend/src/components/ui/Card.tsx
type Props = {
  children: React.ReactNode; // contenu inséré dans la carte
  className?: string; // classe optionnelle pour personnaliser le style
};

// Composant "conteneur" purement présentiel : il n'a aucune logique métier,
// ce qui le rend réutilisable partout (articles, calendrier, classements...).
export function ArticleCard({ children, className = "" }: Props) {
  return (
    <div className={`article-card ${className}`}>
      {children}
    </div>
  );
}
```

Pourquoi ce choix : un composant générique sans logique permet de **réutiliser** la même carte dans toute l'application en ne faisant varier que son contenu et sa classe. Le typage Props (TypeScript) **documente et fiabilise** l'usage du composant (un oubli de propriété est détecté à la compilation).

2. Structure sémantique et accessibilité

Les sections utilisent des balises **HTML sémantiques** (<section>, <article>, titres) et des attributs d'accessibilités.

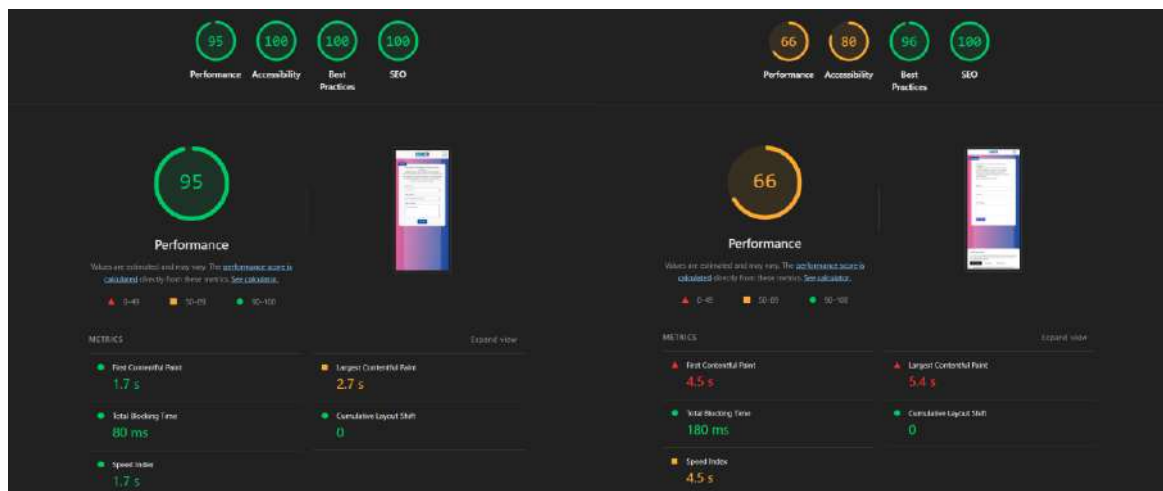
Exemple de section listant les articles d'une discipline :

```
// frontend/src/components/posts/PostsSection.tsx (extrait)
export function PostsSection({ posts, discipline }: PostsSectionProps) {
  return (
    <section id="articles" className="club-posts-section">
      <ArticleTitle>Actualités</ArticleTitle>
      <ArticleCard>
        {posts.length > 0 ? (
          // on parcourt les articles et on délègue l'affichage à un sous-
composant
          posts.map((post) => (
            <ClubPostCard key={post.id} post={post}
              from={`/disciplines/${discipline}/articles`} />
          ))
        ) : (
          // gestion explicite du cas "aucune donnée"
          <p>Aucun article pour cette discipline.</p>
        )}
      </ArticleCard>
    </section>
  );
}
```

3. Responsivité et SEO

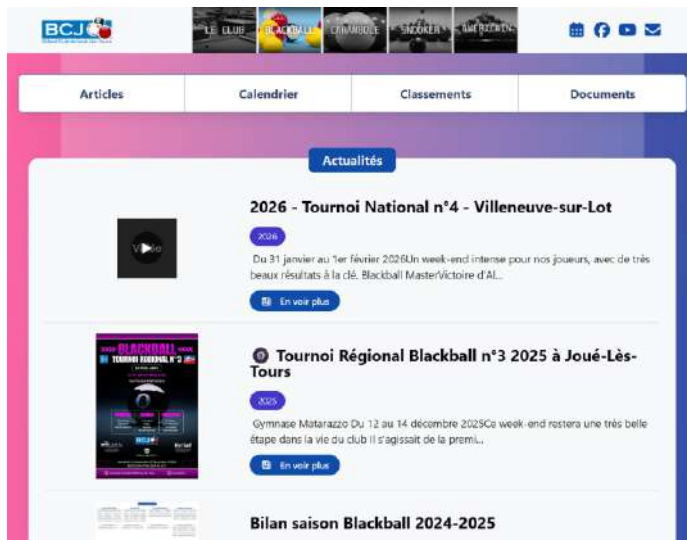
L'adaptation **web / mobile** est assurée par des feuilles de styles responsives (points de rupture) et par des composants de navigation distincts (DesktopNavigation, MobileNavigation). Le référencement est traité par **React Helmet Async** (balises title, meta description, Open Graph), un **sitemap.xml**, un **robots.txt** et des **URLs propres**.

Résultat mesuré (Lighthouse) : performance ~66 (ancien site) -> **~95** ; **accessibilité 100**, **bonnes pratiques 100**, **SEO 100**

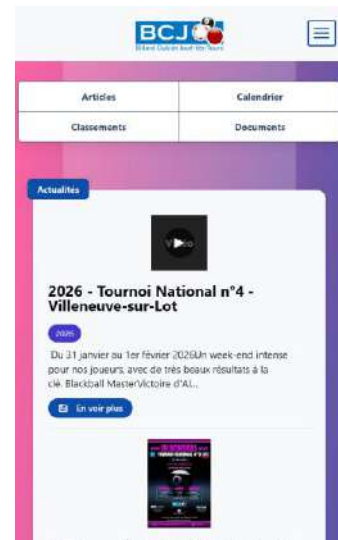


Lighthouse React

Lighthouse Blade



Discipline desktop



Discipline Mobile

4. Partie dynamique des interfaces

a) Consommation de l'API REST

Tous les appels passent par un **client HTTP centralisé et typé**. Il standardise les requêtes, l'ajout des en-têtes et surtout la **gestion des erreurs**, condition d'un style de programmation défensif :

```
// frontend/src/api/client.ts (extrait)
export class ApiError extends Error {
  status: number;
  body: unknown;
  constructor(message: string, status: number, body: unknown = null) {
    super(message);
    this.name = "ApiError";
    this.status = status; // on conserve le code HTTP pour réagir précisément (ex :
404)
    this.body = body;
  }
}

export async function apiGet<T>(endpoint: string): Promise<T> {
  const response = await fetch(`${API_URL}${endpoint}`, {
    headers: { Accept: "application/json" },
  });
  const body = await parseResponseBody(response);

  if (!response.ok) {
    // toute réponse non 2xx déclenche une erreur typée, gérée en amont par les
composants
    throw new ApiError(`API error ${response.status}`, response.status, body);
  }
  return body as T;
}
```

Pourquoi ce choix : centraliser les appels évite la duplication de garantit que **toutes** les erreurs sont traitées de façon homogène. La classe **ApiError** **conserve le code HTTP**, ce qui permet de distinguer un « article introuvable » (404) d'une « panne serveur » (500). Le **type générique <T>** assure que la réponse est typée par l'appelant.

Sécurité : l'URL de l'API provient d'une variable d'environnement (`VITE_API_URL`), aucune valeur sensible n'est codée en dur.

Les fonctions d'accès aux ressources restent ainsi très simples et typées :

```
// frontend/src/api/postsApi.ts
export function getPostBySlug(slug: string) {
  return apiGet<ApiResponse<Post>>(`/posts/slug/${slug}`);
}
export function getAllPosts(page = 1) {
  return apiGet<PaginatedResponse<Post>>(`/posts?page=${page}`);
}
```

b) Chargement asynchrone et gestion des états

La page de détail d'un article illustre le **cycle de vie complet** d'un composant dynamique : chargement asynchrone, gestion des états (chargement / erreur / introuvable / succès) et nettoyage.

```
// frontend/src/pages/PostPage.tsx (extrait)
const [state, setState] = useState<PageState>({
  post: null, loading: true, error: false, notFound: false,
});
useEffect(() => {
  if (!slug) return;
  let isMounted = true; // évite de mettre à jour un composant démonté
  getPostBySlug(slug)
    .then((response) => {
      if (!isMounted) return;
      setState({ post: response.data, loading: false, error: false, notFound: false });
    })
    .catch((error) => {
      if (!isMounted) return;
      // on distingue précisément le 404 (article inexistant) des autres erreurs
      if (error instanceof ApiError && error.status === 404) {
        setState({ post: null, loading: false, error: false, notFound: true });
        return;
      }
      setState({ post: null, loading: false, error: true, notFound: false });
    });
  return () => { isMounted = false; }; // nettoyage au démontage
}, [slug]); // relance si le slug change
```

- **useState** : modélise les quatre états possibles de la page, pas d'affichage incohérent
- **useEffect** : déclenche l'appel API et se **relance** quand l'affichage change [slug]
- Le drapeau **isMounted** et la **fonction de nettoyage** évitent les fuites mémoire et les erreurs de mise à jour après démontage (**style défensif**)
- La branche **error instanceof ApiError && status === 404** affiche une page « introuvable » dédiée au lieu d'une erreur générique, meilleure **expérience utilisateur**.

c) Interactivité, accessibilité et SEO dynamique

Le composant calendrier regroupe les événements par niveau et propose un **accordéon accessible** :

```
// frontend/src/components/calendar/CalendarSection.tsx (extrait)
const [openedEventId, setOpenedEventId] = useState<number | null>(null);
const isOpen = openedEventId === event.id;
```

```
<button type="button" className="calendar-event__header"
  onClick={() => setOpenEventId(isOpen ? null : event.id)}
  aria-expanded={isOpen}>      {/* état annoncé aux lecteurs d'écran */}
  {getEventTitle(event)}
</button>
```

L'attribut **aria-expanded** informe les technologies d'assistance de l'état ouvert/fermé : c'est une réponse concrète au critère « expérience utilisateur, y compris pour les personnes en situation de handicap ».

Le **SEO dynamique** est généré par page à partir des données chargées :

```
// frontend/src/pages/PostPage.tsx (extrait)
<Helmet title={` ${postTitle} - BCJ37`}>
  <meta name="description" content={postDescription} />
  <meta property="og:title" content={` ${postTitle} - BCJ37`} />
  {post.image_url && <meta property="og:image" content={post.image_url} />}
</Helmet>
```

d) Point de sécurité assumé : contenu riche des articles

Le contenu HTML des articles est rendu via **dangerouslySetInnerHTML** car il est **rédigé par les administrateurs du club** dans le back-office (texte enrichi). Le **périmètre de confiance** est donc limité aux comptes administrateurs authentifiés.

Limite identifiée et piste d'amélioration :

Un **assainissement** (sanitization, ex. DOMPurify) côté rendu pour se prémunir d'un contenu malveillant en cas de compromission d'un compte.

IV. Synthèse

Compétences	Démontré par
Maquetter	Maquettes web + mobile + schéma d'enchaînement
Interfaces statiques	Composants réutilisables typés, HTML sémantique, responsive, SEO, Lighthouse
Partie dynamique	Client API typé + ApiError , useState / useEffect , états et nettoyage, accordéon aria-expanded , SEO dynamique Helmet

10) Accessibilité et Expérience Utilisateur (UX)

I. Objectifs

L'accessibilité et l'expérience utilisateur sont des éléments essentiels du projet, notamment dans un contexte associatif visant **inclusion, accessibilité et diversité des publics**.

II. Accessibilité

1. Structure sémantique

Les pages utilisent des éléments HTML adaptés :

- `<main>`

- `<section>`
- `<nav>`
- `<h1>`, `<h2>`

Objectif :

- Faciliter la lecture par les technologies d'assistance (lecteurs d'écran)

2. Attributs ARIA

Exemple d'utilisation :

- `Aria-labelledby`
- `Aria-describedby`
- `role= « region »`
- `role= « alert » / role= « status »`

Ces attributs permettent :

- De donner du contexte aux éléments
- D'améliorer la navigation pour les utilisateurs malvoyants

3. Contenus alternatifs

- Texte alternatif pour la carte (*sr-only*)
- Lien complémentaire vers Google Maps
- Messages explicites pour les erreurs et succès

4. Formulaire accessible

Le formulaire de contact respecte plusieurs bonnes pratiques :

- Association label / input
- Champs obligatoires
- Messages d'erreur compréhensibles
- Gestion des états (envoi, succès, erreur)

III. Expérience Utilisateur (UX)

1. Navigation claire

- Menu principal structuré
- Sous-menu dynamique dans les disciplines
- Navigation contextuelle (retour article -> discipline du club)

2. Organisation des contenus

Les pages sont organisées en sections logiques :

- Articles
- Classements
- Calendrier
- Documents

Objectifs :

- Faciliter la compréhension
- Guider l'utilisateur
- Eviter la surcharge d'information

3. Feedback utilisateur

Le système fournit des retours visuels :

- Message de succès (contact)
- Message d'erreur
- Etat de chargement

4. Performance perçue

- Chargement dynamique des données
- Affichage conditionnel
- Limitation des rechargements de page

5. Intégration de contenus externes

- Carte interactive (OpenStreetMap)
- Calendrier Google
- Classement PDF

Ces éléments enrichissent l'expérience utilisateur sans complexifier l'interface.

11) Réalisation Back-end

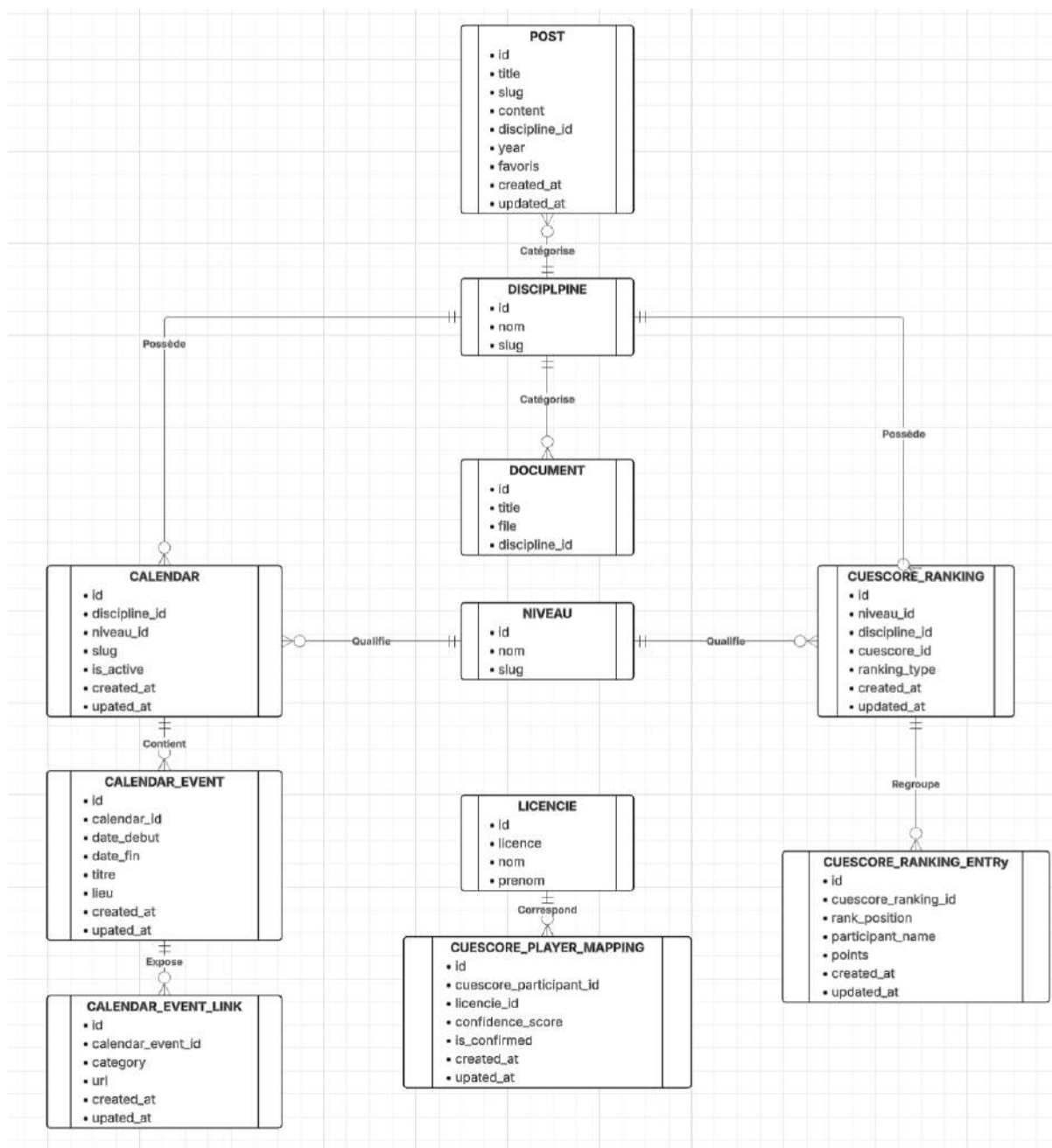
Le back-end est une **API-Rest Laravel 10 (PHP 8.3)** sur base **MySQL**. Il expose les données au front-end React, héberge la logique métier et alimente l'administration OpenAdmin. Toutes les réponses respectent un **format standardisé : data / meta / links / error**.

I. Mise en place de la base de données relationnelle

1. Du modèle conceptuel au modèle physique

La conception part de l'expression des besoins et suit la démarche **MCD -> MLD -> schéma physique**

- **MCD**



Entités **Discipline**, **Niveau**, **Post**, **Document**, **Calendar**, **CalendarEvent**, **CalendarEventLink**, **Licencie**, **CueScoreRanking**, **CueScoreRankingEntry**, **CueScorePlayerMapping** et leurs relations.

- **MLD / Schéma physique**, traduction en tables, clés primaires, clés étrangères et clé uniques.
- **Script de création** : script SQL complet, testé sur MySQL 8.

2. Script de création de la base

A partir du schéma physique, la base est créée par un script SQL. L'extrait ci-dessous présente les tables qui portent la **normalisation**. Le script complet a été **validé sur MySQL 8** (13 tables, intégrité garantie par les clés étrangères).

```
-- Tables de référence : les disciplines et les niveaux de compétition
CREATE TABLE `discipline` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `nom` VARCHAR(100) NOT NULL,
  `slug` VARCHAR(100) NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_discipline_slug` (`slug`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `niveau` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `nom` VARCHAR(100) NOT NULL,
  `slug` VARCHAR(100) NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_niveau_slug` (`slug`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- CALENDAR : un calendrier = une discipline x un niveau
-- (résolution de la relation N:N entre DISCIPLINE et NIVEAU)
CREATE TABLE `calendar` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `discipline_id` INT UNSIGNED NOT NULL,
  `niveau_id` INT UNSIGNED NOT NULL,
  `name` VARCHAR(255) DEFAULT NULL,
  `slug` VARCHAR(255) NOT NULL,
  `is_active` TINYINT(1) NOT NULL DEFAULT 1,
  `created_at` TIMESTAMP NULL DEFAULT NULL,
  `updated_at` TIMESTAMP NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_calendar_slug` (`slug`),
  UNIQUE KEY `uk_calendar_discipline_niveau` (`discipline_id`, `niveau_id`),
  CONSTRAINT `fk_calendar_discipline`
    FOREIGN KEY (`discipline_id`) REFERENCES `discipline` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_calendar_niveau`
    FOREIGN KEY (`niveau_id`) REFERENCES `niveau` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Données de référence : les chaînes « magiques » deviennent des lignes référencées
INSERT INTO `discipline` (`id`, `nom`, `slug`) VALUES
(1, 'Blackball', 'blackball'), (2, 'Carambole', 'carambole'),
(3, 'Snooker', 'snooker'), (4, 'Américain', 'americain');

INSERT INTO `niveau` (`id`, `nom`, `slug`) VALUES
(1, 'International', 'international'), (2, 'National', 'national'),
(3, 'Régional', 'regional'), (4, 'Départemental', 'departemental');
```

3. Règles appliquées

Critère	Mise en œuvre BCJ37
Schéma conforme + règles du relationnel	Tables normalisées, aucune donnée répétée
Règle de nommage	Snake_case, table au singulier métier, suffixe _id, contraintes préfixés fk_* / uk_*
Intégrité	Clé étrangères avec ON DELETE CASCADE / SET NULL

Sécurité / confidentialité	Compte MySQL applicatif à droits limités. Données personnelles des licenciés isolées (RGPD)
Base de tests conforme	Migrations Laravel + base de test (script validé sur MySQL 8)
Sauvegarde / restauration	Export régulier (dump) restaurable en cas d'incident

II. Composants d'accès aux données

L'accès aux données utilise l'**ORM Eloquent**, qui produit des **requêtes paramétrées** (protection native contre l'**injection SQL**). Les modèles centralisent relations, filtres et règles de cohérence.

1. Modèle et relation

```
// app/Models/Calendar.php (extrait)
class Calendar extends Model
{
    protected $fillable = ['discipline', 'scope', 'name', 'slug', 'source_type',
    'is_active'];

    // typage automatique : 'is_active' est exposé en booléen, pas en 0/1
    protected $casts = ['is_active' => 'boolean'];

    // un calendrier possède plusieurs événements, triés par date
    public function events(): HasMany
    {
        return $this->hasMany(CalendarEvent::class, 'calendar_id')
            ->orderBy('date_debut', 'asc');
    }

    // filtre réutilisable : ne renvoyer que les calendriers actifs
    public function scopeActive(Builder $query): Builder
    {
        return $query->where('is_active', true);
    }
}
```

Pourquoi ces choix :

- **\$fillable** liste explicitement les champs assignables en masse -> protection contre le *mass-assignment* (un attaquant ne peut pas injecter un champ non prévu)
- Les **\$casts** garantissent des types cohérents côté API (booléens, dates)
- Les **relations** (**hasMany**, **belongsToMany**) et les **query scopes** (**scopeActive**, **scopeByDiscipline**, ...) factorisent les filtres et évitent la duplication de requêtes.

Le modèle **CalendarEvent** complète l'arbre relationnel (**belongsToMany Calendar**, **hasMany CalendarEventLink**) et expose des **accessors** calculés (**is_upcoming**, **is_past**) ainsi que les scopes métier (**scopeUpcoming**, **scopeBetweenDates**).

2. Validation des entrées et format de réponse

Le contrôleur d'API **valide systématiquement** les paramètres avant toute requête, puis renvoi le format standardisé :

```
// app/Http/Controllers/Api/CalendarController.php (extrait)
public function byDiscipline(string $discipline): JsonResponse
{
    // on rejette une discipline inconnue AVANT d'interroger la base
    $this->abortIfInvalidDiscipline($discipline); // → 404 si invalide
}
```



```

$calendars = Calendar::query()
    ->active()           // query scope : calendriers actifs
    ->byDiscipline($discipline)
    ->ordered()
    ->withCount('events') // évite le problème N+1 (compte les événements en une
requête)
    ->get();
return response()->json([
    'data' => CalendarResource::collection($calendars),
    'meta' => ['discipline' => $discipline, 'count' => $calendars->count()],
    'links' => [],
    'error' => null,
]);
}
private function abortIfInvalidDiscipline(string $discipline): void
{
    abort_unless(Calendar::isValidDiscipline($discipline), 404, 'Discipline invalide.');
```

- **Validation en amont (abort_unless)** : aucune requête n'est lancée avec un paramètre non maîtrisé
- **withCount** : optimisation des accès (évite le problème **N+1**)
- **CalendarResource** : couche de transformation qui contrôle exactement les champs exposés (pas de fuite de données internes)
- **Format de réponse homogène (data/meta/links/error)** consommé tel quel par le front.

3. Aspect « non relationnel »

Le projet est relationnel, mais il consomme l'**API externe CueScore** dont les réponses sont en **JSON** (semi-structuré). Le champ **source_payload** (type **JSON** MySQL, casté en tableau PHP) conserve la charge utile brute des événements importés.

III. Composants métier côté serveur

1. Un composant métier simple et testable

Le mapping discipline -> identifiant est isolé dans une classe dédiée, pure et facilement testable :

```

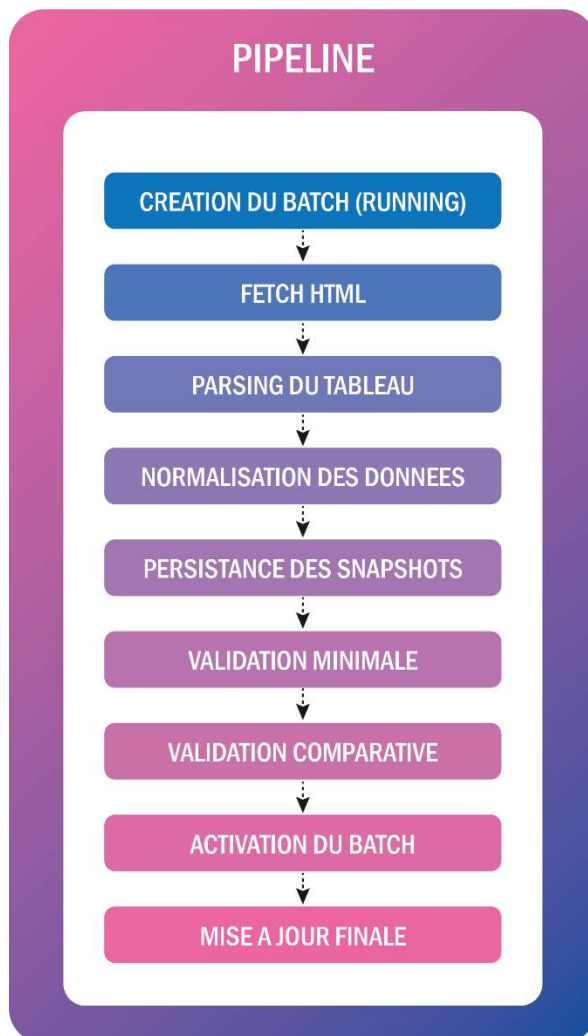
// app/Support/DisciplineMapper.php (extrait)
class DisciplineMapper
{
    public const DISCIPLINES = ['blackball' => 1, 'carambole' => 2, 'snooker' => 3,
'américain' => 4];
    public static function idFromSlug(string $slug): ?int
    {
        return self::DISCIPLINES[$slug] ?? null; // null si slug inconnu : pas
d'exception non maîtrisée
    }
    public static function isValidSlug(string $slug): bool
    {
        return array_key_exists($slug, self::DISCIPLINES);
    }
}
```

Pourquoi ce choix :

Centraliser cette correspondance évite de la disperser dans le code (principe **DRY**) et fournit un point unique de validation (**isValidSlug**).

2. Architecture métier avancée : le pipeline d'import des licenciés

La fonctionnalité la plus représentative côté serveur est l'**import automatisé des licenciés** depuis la plateforme fédérale Telemat. Elle est organisée en **architecture multicouche** (Domain-Driven Design) sous **app/Domain/LicenseImport**, séparée en étapes : **Fetch -> Parsing -> Normalization -> Snapshot -> Validation -> Activation -> Projection -> Reporting**



L'**orchestrateur** enchaîne ces étapes et gère chaque échec de façon défensive :

```
// app/Domain/LicenseImport/Pipeline/TelematLicenseImportOrchestrator.php (extrait)
final class TelematLicenseImportOrchestrator
{
    // toutes les dépendances sont injectées (inversion de dépendances) et en lecture seule
    public function __construct(
        private readonly TelematFetcherInterface $httpClient,
        private readonly TelematHtmlTableParser $htmlTableParser,
        private readonly TelematRowNormalizer $rowNormalizer,
        private readonly TelematMinimalBatchValidator $minimalBatchValidator,
        private readonly TelematComparativeBatchValidator $comparativeBatchValidator,
        private readonly TelematBatchActivator $batchActivator,
        private readonly TelematFailureManager $failureManager,
        // ... (autres collaborateurs)
    ) {}
    public function run(LicenseImportExecutionContext $context): LicenseImportBatch
    {
        $batch = $this->batchLifecycleManager->start($context);
    }
}
```

```
try {
    $fetchResult = $this->dispatchFetch($context);
    $parsedTable = $this->htmlTableParser->parse($fetchResult->html, $context);
    $normalizedRows = $this->rowNormalizer->normalize($parsedTable, $context);
    $this->minimalBatchValidator->validate($batch, $context); // garde-fou n°1
    $this->comparativeBatchValidator->validate($batch, $context); // garde-fou n°2
    $this->batchActivator->activate($batch, $context);
    return $batch;
} catch (MinimalValidationFailedException $e) {
    // chaque type d'erreur est tracé puis relancé : aucune défaillance silencieuse
    $this->failureManager->handleStageFailure($batch, $context, $e,
        'minimal_validation', 'minimal_validation_failed');
    throw $e;
} catch (Throwable $e) {
    $this->failureManager->handleUnexpectedFailure($batch, $context, $e);
    throw $e;
}
}
```

- **Bonnes pratiques POO**: classe **final**, propriétés **readonly**, **injection de dépendances**, **declare(strict_types=1)**
- **Architecture multicouche** : chaque étape est une classe à responsabilité unique (parsing, normalisation, validation ...), ce qui facilite les tests et la maintenance (**refactoring** sûr)
- **Style défensif : deux niveaux de validation** (minimale puis comparative) avant d'activer un lot, et **capture typée des exceptions** par étape, aucune erreur n'est silencieuse, chaque échec est **journalisé**
- **Sécurité des données** : un lot n'est activé qu'après validation, la **validation comparative** empêche d'écraser les licenciés existants par un import incohérent (ex. fichier tronqué).

3. Fonction scraping / import des licenciés

e) Objectif du système d'import

La fonctionnalité de scraping n'est pas utilisée comme une fonction d'affichage direct, mais comme un système d'import métier exécuté en arrière-plan.

Son objectif est de :

- Récupérer automatiquement les licenciés depuis une source externe
- Fiabiliser les données
- Détecter les anomalies
- Historiser les imports
- Publier uniquement les données validées

Dans ce projet, les données proviennent du site de la Fédération Française de Billard.
<https://www.telemat.org/FFBI/sif/?cs=4.4e8caa4669b605602da29fce0147c7038d4c>

Licences et pass billard scolaire

Numéro Nom Prénom Club

Licences du club 15061 – BILLARD CLUB DE JOUE LES TOURS

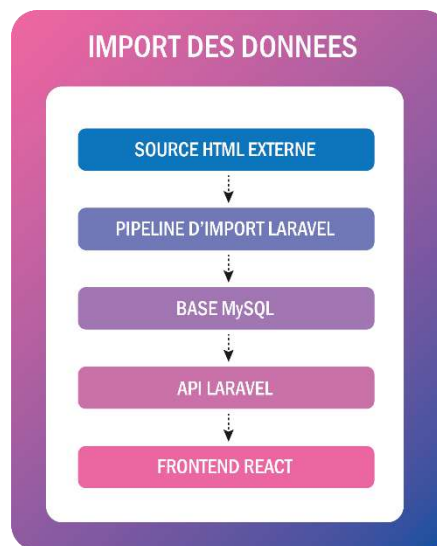
Numéro	Nom	Prénom	Catégorie
191100 S	ARROUAS	ISABELLE	Decouverte
190399 F	AUCHART	THIERRY	Decouverte
018952 Y	AUGER	WILLIAM	+21 ans
185211 R	BAILLON	DIDIER	+21 ans
126663 R	BARBIER	JEAN FRANÇOIS	+21 ans
165139 Z	BARICAULT	JEAN CHRISTOPHE	+21 ans
173547 N	BARRAS	MICHEL	+21 ans
111666 W	BONNET	MICHEL	+21 ans
188587 L	BOUATTAOUN	TARIK	+21 ans
177407 J	BOURLIER	ALEXANDRE	+21 ans
137219 R	BOUTEILLE	MATHEO	-21 ans
180469 M	BOUYCHOU	CYRIL	+21 ans
148836 D	BRUNEAU	MARTIN	+21 ans
141625 D	BUSCETTI	ALEXANDRE	+21 ans
179777 K	BUZIAUX	NANCY	+21 ans
157681 S	CADINOT	GREGORY	+21 ans
174450 V	CARVALHO	RAUL	+21 ans
183753 G	CESVRE	JEAN LOUIS	+21 ans
183253 N	CHAS	THIERRY	+21 ans
131348 W	CHAUVIN	ROLAND	+21 ans

Source externe utilisée pour récupérer automatiquement les licenciés du club

f) Positionnement dans l'architecture globale

Le frontend React ne communique jamais directement avec la source externe.

L'import des données est réalisé côté serveur via Laravel.



Cette architecture permet :

- Le découplage complet du frontend
- Une meilleure stabilité
- Une validation des données avant publication
- Une historisation des imports

g) Fonctionnement global du pipeline

Chaque import suit plusieurs étapes successives :

Pipeline complet d'import des licenciés depuis la récupération HTML jusqu'à la publication des données dans l'API

h) Import batch et validation

Chaque récupération crée un batch d'import indépendant.

Le pipeline :

- Récupère les données
- Analyse le HTML
- Normalise les colonnes
- Compare les résultats avec l'import précédent
- Décide si les données peuvent être publiées

Afin d'éviter qu'un mauvais scraping écrase des données saines, plusieurs états sont utilisés :

Etat	Rôle
RUNNING	Le batch est en cours d'exécution
SUCCEEDED	Le parsing et les validations techniques sont terminés
HOLD	Le batch est conservé mais placé en quarantaine
REJECTED	Le batch est refusé car il persiste trop d'anomalies
FAILED	Une erreur technique a interrompu le processus
ACTIVATED	Le batch est validé et devient la source publiée

```

<?php

namespace App\Domain\LicenseImport\Enums;

enum BatchStatus: string
{
    case Pending = 'pending';
    case Running = 'running';
    case Fetched = 'fetched';
    case Parsed = 'parsed';
    case ValidatedMinimal = 'validated_minimal';
    case ValidatedComparative = 'validated_comparative';
    case Activated = 'activated';
    case Rejected = 'rejected';
    case Failed = 'failed';
}

```

Les états du batch permettent de sécuriser la publication des données et d'éviter qu'un import invalide remplace les données existantes.

i) Organisation technique Laravel

Couche	Rôle
FETCH	Récupération HTML
PARSING	Lecture des tableaux HTML
NORMALISATION	Nettoyage et mapping des données
VALIDATION	Contrôle qualité
ACTIVATION	Publication des données
PIPELINE	Orchestration globale

Cette architecture améliore :

- La maintenabilité
- La testabilité
- La robustesse
- L'évolutivité du système

Domain

```

└─ LicenseImport
   │ └─ Fetch
   │ └─ Parsing
   │ └─ Normalization
   │ └─ Validation
   │ └─ Activation
   └─ Pipeline
  
```

Organisation du domaine Laravel responsable du pipeline d'import des licenciés

j) Gestion des erreurs et tests

Le pipeline distingue plusieurs types d'erreurs :

- Erreurs réseau
- Erreurs de parsing
- Erreurs de validation
- Erreurs d'activation

Chaque erreur :

- Génère un rapport
- Conserve les données pour analyse
- Empêche la publication d'un import invalide

Des tests :

- Unitaires
- D'intégration
- Fonctionnels

Ont été mis en place afin de sécuriser les traitements.

k) Cas concret d'utilisation

Les données importées sont ensuite utilisées dans :

- Les pages React
- Les classements
- Les licenciés du club
- L'administration OpenAdmin

Le frontend React consomme ensuite les données via l'API Laravel sans dépendre directement de la source externe.

National		
FFB - Blackball - TN - Handi-Billard - 2025-2026		
2ème	Thomas RABAUD	140 points
3ème	Christophe FERRASSE	112 points
4ème	Gregory Cédric	88 points
FFB - Blackball - TN - Master - 2025-2026		
5ème	Christophe Lambert	1776 points
7ème	Alexandre Baccetti	1600 points
12ème	Paul Collinck	1000 points
15ème	Elié Chénard	1014 points
16ème	Jean-Jacques	1000 points
FFB - Blackball - TN - Mixte National - 2025-2026		
9ème	Jerome Landroen	692 points
29ème	Antoine Stenier	307 points
30ème	Christophe Faure	302 points
33ème	Nicolas DUBOIS	518 points
57ème	Capitain Laval	447 points
FFB - Blackball - TN - Vétérans - 2025-2026		
8ème	Jerome Landroen	532 points
FFB - Blackball - TN - Féminin - 2025-2026		
5ème	Capitain Laval	584 points
FFB - Blackball - TN - Espoir - 2025-2026		
24ème	Mathis Baccetti	60 points
FFB - Blackball - TN - Junior - 2025-2026		
13ème	Colin Des Cordes	180 points
FFB - Blackball - Equipes - DNF - 2025-2026		
1er	VERDUN 1 DNF	100 points
2ème	JOUE LES TOURES 1 DNF	80 points
3ème	LES HERBIBUS DNF	76 points
4ème	JOUE LES TOURES 2 DNF	76 points
5ème	MEC LE COURE DNF	66 points
6ème	QUIMPER DNF	64 points
7ème	VERDUN 2 DNF	60 points
8ème	DOUVEQUE DNF	58 points
9ème	ASBEN DNF	37 points
10ème	SAINT JEAN DE PORT DNF	48 points
11ème	ASBEN DNF	47 points
12ème	NOYAL PONTIVY DNF	38 points

Exemple d'exploitation des données importées dans le frontend React

l) Bilan

Cette architecture permet :

- Une forte séparation des responsabilités
- Une meilleure robustesse face aux changements HTML
- Une historisation des imports
- Une publication sécurisée des données
- Une intégration propre avec Laravel API et React.

Le système d'import devient ainsi une véritable couche métier indépendante et non un simple script de scraping.

IV. Synthèse

Compétences	Démontré par
Base de données	MCD normalisé, MLD, script, règles de nommage et d'intégrité.
Accès aux données	Eloquent (relations, scopes, cats, \$fillable), validation des entrées, requêtes paramétrées, format data/meta/links/error , JSON CueScore
Composants métier	DisciplineMapper + pipeline DDD LicenseImport (POO, multicouche, style défensif, exceptions typées)

12) Sécurité de l'application

I. Mesures mises en place

1. Authentification API et protection des routes sensibles

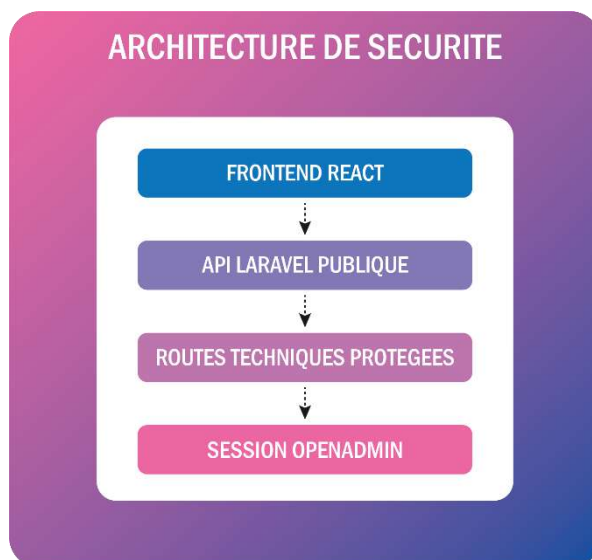
Le projet BCJ37 distingue deux types de routes :

- Les routes publiques destinées au frontend React
- Les routes techniques et administratives réservées à OpenAdmin

Les routes publiques de consultation (posts, documents, calendrier, disciplines, etc) restent accessibles sans authentification afin d'alimenter le frontend React.

En revanche, les routes internes liées aux traitements d'importation et à l'administration sont protégées par un middleware personnalisé basé sur la session OpenAdmin.

Architecture de sécurité utilisée :



Les routes sensibles utilisent :

- Le middleware web
- Un middleware API personnalisé **admin.api**

Ce middleware vérifie qu'un administrateur OpenAdmin est authentifié avant d'autoriser l'accès aux endpoints techniques.

En cas d'accès non autorisé, l'API retourne une réponse JSON standardisée :

```
1  {  
2    |  "message": "Unauthenticated."  
3  }
```

Cette approche permet :

- D'éviter la duplication d'authentification
- De réutiliser la session Laravel existante
- De conserver une architecture API cohérente
- De sécuriser les traitements internes

2. CORS restrictif

Le frontend React et l'API Laravel étant découplés, une configuration CORS (Cross-Origin Resource Sharing) a été mise en place.

Cette configuration limite les origines autorisées à communiquer avec l'API.

Exemple :

Frontend React : <http://localhost:5173>

API Laravel: <http://localhost:8000/api/v1>

Les règles CORS autorisent uniquement :

- Les domaines prévus
- Les méthodes http nécessaires
- Les en-têtes autorisés.

Cette configuration empêche des applications externes non prévues d'effectuer des requêtes navigateur vers l'API

3. Rate limiting

Un système de limitation de requêtes (rate limiting) a été mis en place sur certaines routes sensibles.

Les routes concernées incluent notamment :

- Le formulaire de contact
- Certaines routes techniques
- Les endpoints potentiellement exposés au spam.

Laravel utilise le middleware **throttle**

Exemple :

```
// Routes pour les contacts
Route::prefix('/contact')->group(function () {
    Route::get('/', [ContactController::class, 'index']);
    Route::post('/', [ContactController::class, 'send'])
        ->middleware('throttle:5,1');
});
```

Cette configuration limite :

- Le nombre de requêtes
- Les tentatives automatiques
- Les abus
- Certains comportements malveillants

Le rate limiting améliore également la stabilité globale de l'API.

II. Intérêt des mesures

Les mesures de sécurité mises en place permettent de sécuriser l'architecture découplée React / Laravel du projet BCJ37.

Elles apportent plusieurs avantages :

- Protection des traitements administratifs
- Limitation des accès non autorisés
- Séparation claire entre frontend public et administration
- Contrôle des requêtes navigateur
- Réduction des risques d'injection
- Limitation des abus automatisés
- Amélioration de la stabilité backend
- Sécurisation des échanges API

L'ensemble contribue à rendre l'application plus robuste et plus adaptée à une mise en production.

III. Conclusion

La sécurité du projet BCJ37 repose sur plusieurs couches complémentaires :

- Sécurisation des routes sensibles
- Contrôle des accès administrateurs
- Validation systématique des données
- Limitation des requêtes abusives
- Protection du frontend React
- Configuration CORS adaptée à une architecture découplée

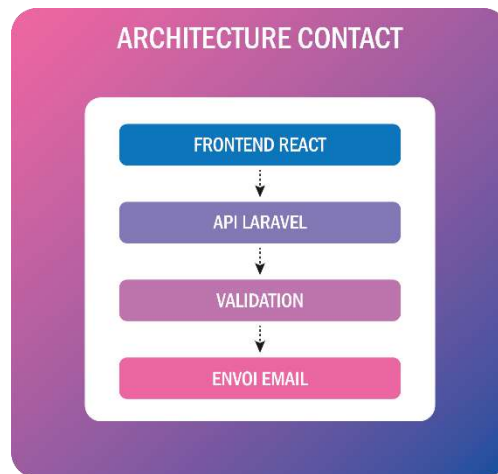
Cette approche permet de conserver :

- Une API moderne
- Un frontend indépendant
- Une administration sécurisée
- Une architecture maintenable et évolutive

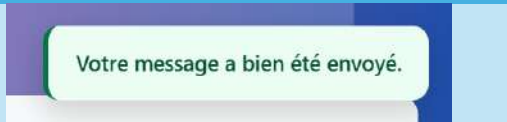
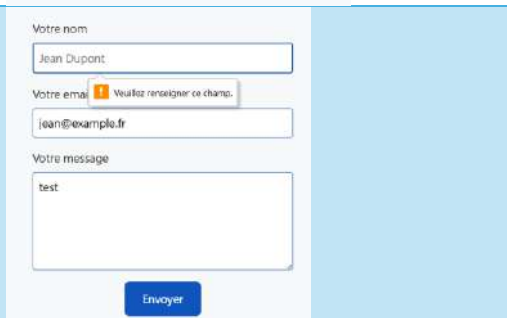
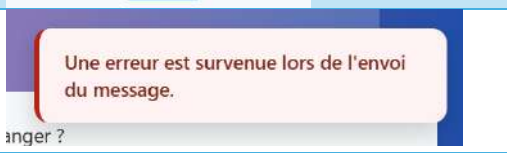
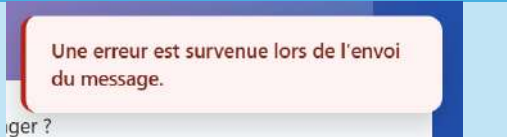
L'application dispose désormais d'une base technique suffisamment robuste pour une utilisation en environnement de production.

13) Fonctionnalité testée

Envoi d'un message via le formulaire de contact.



Critères de réussite : seules des données valides déclenchent l'envoi. Toute entrée invalide ou malveillante est rejetée avec un message clair, sans erreur serveur non maîtrisée.

#	Cas	Données en entrée	Données attendues	Données obtenues
1	Nominal	Name = « Jean Dupont », « email = « jean@example.fr », message = « Bonjour, ... »	HTTP 200, error:null, mail reçu	
2	Email invalide	Name = « Jean », email = « jean@@ », message = « test »	HTTP 422, erreur de validation sur « email », aucun mail	
3	Champ requis manquant	Name = « », email = « jean@exemple.fr », message = « test »	HTTP 422, erreur « name requis », aucun mail	
4	Injection HTML / XSS	Name = <script>alert(1)</script>, email = « jean@example.fr », message = « test »	HTTP 422, rejet par la règle not_regex, aucun mail	
5	Dépassement de taille	Message = chaîne de 4000 caractères.	HTTP 422, erreur max:3000, aucun mail	

Analyse des écarts :

Pour chacun des 5 cas les **données obtenues sont identiques aux données attendues : aucun écart n'a été constaté**. Le comportement est donc conforme. La validation serveur bloque les entrées invalides et malveillantes (cas 2 à 5) et seul le cas nominal aboutit à l'envoi du message. Ce résultat démontre le style défensif du composant (validation systématique, règle **not-regex** anti-injection, gestion d'erreur sans fuite d'information).



Résultat W3C pour la page Contact

14) Veille sur la sécurité

I. Analyse des principales vulnérabilités OWASP et mesures préventives

Dans le cadre de la sécurisation du projet BCJ37, une veille a été réalisée à partir des principales catégories de risques identifiées par l'OWASP Top 10.

L'objectif était d'identifier les vulnérabilités les plus pertinentes pour une architecture découplée composée d'un frontend React, d'une API Laravel, d'une base de données MySQL et d'une administration OpenAdmin.

Contrôle d'accès défaillant

Le risque principal concerne l'accès non autorisé à des routes sensibles.

Dans BCJ37, les routes publiques de consultation restent accessibles sans authentification. En revanche, les routes techniques liées aux imports et aux traitements internes sont protégées par un middleware dédié.

Les tests locaux ont confirmé qu'un utilisateur non authentifié reçoit une réponse :

```
1 {
2   "message": "Unauthenticated."
3 }
```

Cette mesure permet de limiter l'accès aux fonctionnalités internes de l'application.

Mauvaise configuration de sécurité

Une mauvaise configuration peut exposer des informations sensibles ou ouvrir l'application à des accès non prévus.

Plusieurs points ont été vérifiés :

- Désactivation du mode debug en production
- Configuration CORS restrictive
- Suppression des routes de test inutiles
- Réponses JSON maîtrisées
- Séparation entre routes publiques et routes protégées

La configuration CORS a notamment été corrigée afin de ne plus autoriser toutes les origines avec *

Injection

Les risques d'injection concernent principalement les données envoyées par les utilisateurs

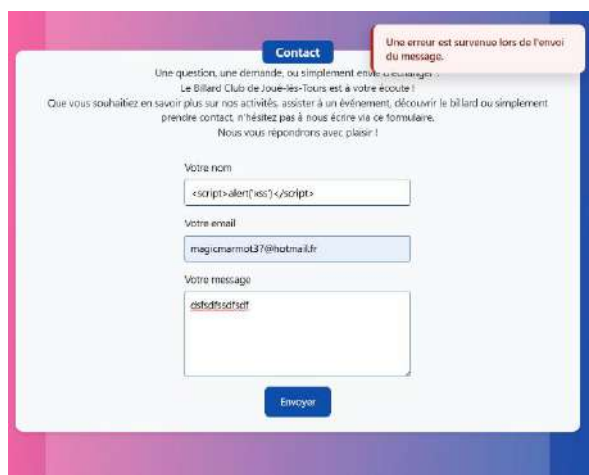
Le projet utilise les mécanismes Laravel :

- Validation des champs
- Requêtes Eloquent
- Query Builder
- Typage des paramètres
- Contraintes sur les routes

Les champs du formulaire de contact sont validés avant traitement, ce qui limite les entrées invalides ou malveillantes.

Cross-Site Scripting

Un test XSS a été effectué avec une balise script dans le formulaire de contact :



The screenshot shows a contact form titled 'Contact' with a blue button. The form contains three input fields: 'Votre nom', 'Votre email', and 'Votre message'. The 'Votre nom' field contains the text '<script>alert('xss')</script>'. The 'Votre email' field contains 'magicmarmot37@hotmail.fr'. The 'Votre message' field contains 'esabdfnschsetf'. A blue 'Envoyer' button is at the bottom. A red error message box at the top right says 'Une erreur est survenue lors de l'envoi du message.' The form is set against a light blue background with a pink and blue gradient border.

La validation Laravel bloque cette entrée. Le script n'est pas exécuté et le message n'est pas envoyé.

Cette vérification confirme que les entrées utilisateurs ne sont pas injectées directement dans l'interface.

Composants vulnérables ou obsolètes

Une veille a également été réalisée sur les dépendances du projet.

Les commandes suivantes ont été réalisées :

```
magic@Alex-PC MINGW64 /c/laragon/www/BCJ (feature/security)
$ composer audit
```

```
magic@Alex-PC MINGW64 /c/laragon/www/BCJ/frontend (feature/security)
$ npm audit
found 0 vulnerabilities
```

Côté frontend, **npm audit** n'a détecté aucune vulnérabilité connue.

Côté backend, **composer audit** a d'abord signalé plusieurs vulnérabilités. Les dépendances concernées ont été mises à jour ou supprimées lorsqu'elles n'étaient pas utilisées.

Après correction, le résultat obtenu est :

```
magic@Alex-PC MINGW64 /c/laragon/www/BCJ (feature/security)
$ composer audit
No security vulnerability advisories found.
```

Limitation des requêtes abusives

Le formulaire de contact étant public, il peut être exposé au spam ou aux envois automatisés.

Un middleware de rate limiting a été ajouté afin de limiter le nombre de requêtes autorisées sur une période donnée.

Lors des tests locaux, après plusieurs envois rapides, l'API a retourné :

```
magic@Alex-PC MINGW64 /c/laragon/www/BCJ (feature/security)
$ curl -X POST http://localhost:8000/api/v1/contact \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-d '{"name":"Test","email":"test@test.fr","message":"Message test"}'
{
  "message": "Too Many Attempts."
}
```

II. Synthèse de la veille sécurité

La veille sécurité a permis de vérifier que le projet BCJ37 prend en compte les principaux risques OWASP adaptés à son architecture.

Cette démarche a permis d'améliorer la sécurité globale du projet avant sa finalisation.

Elle montre également que la sécurité ne repose pas sur une seule mesure, mais sur plusieurs couches complémentaires.

En conclusion, cette veille a permis de renforcer l'application sur les aspects les plus critiques : accès, configuration, validation, dépendances et abus automatisés.

Le projet BCJ37 dispose ainsi d'une base de sécurité cohérente pour une mise en production.

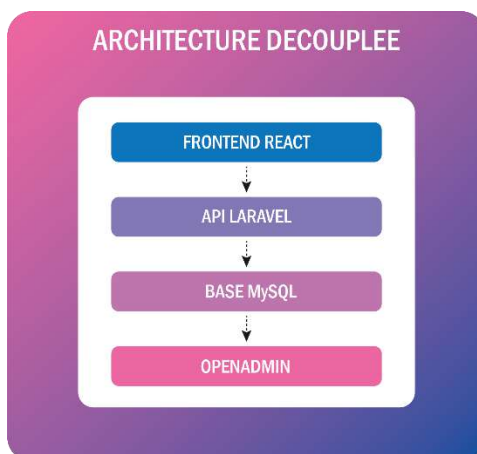
15) Déploiement

I. Mise en production – introduction

La mise en ligne du projet BCJ37 a été réalisée sur un environnement de préproduction hébergé sur le serveur **test.alexandrebourlier.fr**.

L'objectif était de valider le fonctionnement complet de l'application dans un environnement distant proche d'une mise en production réelle.

Le déploiement repose sur une architecture découplée.



Le frontend React est servi depuis le domaine de test, tandis que l'API Laravel est isolée sur un sous-domaine dédié.

II. Préparation de l'hébergement

1. Création des sous-domaines

Deux espaces ont été configurés sur le serveur :
Test.alexandrebourlier.fr

Pour le frontend React,

Et :

Api.test.alexandrebourlier.fr

Pour le backend Laravel, l'API REST et l'administration OpenAdmin.

Cette séparation permet de conserver une architecture claire :

- Le frontend React affiche l'interface utilisateur
- Laravel expose les données via l'API
- OpenAdmin reste rattaché au backend Laravel

2. Certificats SSL

Des certificats SSL Let's Encrypt ont été activés sur les domaines utilisés :

api.test.alexandrebourlier.fr	www.api.test.alexandrebourlier.fr	Installé	http-01	18 Aug 2026	Supprimer Réinstaller Voir
test.alexandrebourlier.fr	www.test.alexandrebourlier.fr test.alexandrebourlier.fr	Installé	http-01	11 Aug 2026	Supprimer Réinstaller Voir

La validité HTTPS a été vérifiée depuis le navigateur, notamment via l'onglet **Security** dans DevTools.

L'API et l'administration sont donc accessibles via HTTPS.

3. Arborescence des dossiers

Le frontend React et le backend Laravel ont été séparés dans deux dossiers distincts.

Arborescence simplifiée :

```
/home/boal2619/
├── test.alexandrebourlier.fr/
│   ├── index.html
│   ├── assets/
│   └── .htaccess
└── api.test.alexandrebourlier.fr/
    ├── app/
    ├── bootstrap/
    ├── config/
    ├── database/
    ├── public/
    ├── resources/
    ├── routes/
    ├── storage/
    ├── vendor/
    ├── artisan
    └── .env
```

Le sous-domaine **api.test.alexandrebourlier.fr** pointe vers le dossier :

/api.test.alexandrebourlier.fr/public

Afin de ne rendre accessible publiquement que le point d'entrée Laravel.

III. Préparation de la base de données

Une base de données MySQL dédiée a été créée sur le serveur.

Les informations de connexion ont été configurées dans le fichier **.env** Laravel :

```
DB_CONNECTION=mysql
DB_HOST=melon.o2switch.net
```

```
DB_PORT=3306
DB_DATABASE=boal2619_test
DB_USERNAME=boal2619
DB_PASSWORD=*****
```

Un problème de cache Laravel a été rencontré : après modification du fichier **.env**, l'application continuait d'utiliser les anciens identifiants de base de données.

Le problème venait du cache de configuration Laravel et d'un ancien fichier **.env.production**.

La résolution a consisté à :

```
rm -f bootstrap/cache/config.php
php artisan optimize:clear
php artisan config:clear
php artisan config:cache
```

La bonne configuration a ensuite été vérifiée avec Tinker :

Php artisan tinker

Puis :

```
config('database.connections.mysql.username')
config('database.connections.mysql.database')
config('database.connections.mysql.host')
```

IV. Déploiement du backend

Le projet Laravel a été copié dans le dossier du sous-domaine API :

Api.test.alexandrebourlier.fr

Les dépendances PHP ont été installées avec Composer :

Composer install - --no-dev

Les dépendances de développement ont été exclues afin d'alléger l'installation en environnement serveur.

Certaines erreurs ont été rencontrées lors de l'installation, notamment avec une configuration résiduelle liée à Scribe. Le fichier de configuration concerné a été supprimé car Scribe était uniquement utilisé pour la documentation en développement.

Les commandes Laravel suivantes ont ensuite été utilisées :

```
php artisan optimize:clear
php artisan config:cache
php artisan storage:link
```

Le lien symbolique **storage:link** permet de rendre accessible les fichiers publics stockés dans :

storage/app/public

Notamment :

- Images des articles
- Miniatures
- Images du menu
- Documents publics

V. Déploiement du frontend (React)

Le frontend React a été préparé avec Vite.

Le fichier **.env.production** du frontend contient l'URL de l'API :

VITE_API_URL=https://api.test.alexandrebourlier.fr/api/v1

Le build de production a été généré avec :

Npm run build

Le contenu du dossier **dist/** a ensuite été copié directement dans :

Test.alexandrebourlier.fr

Un fichier **.htaccess** a été ajouté pour permettre le bon fonctionnement de React Router :

RewriteEngine On

RewriteCond %{REQUEST_FILENAME} -f [OR]

RewriteCond %{REQUEST_FILENAME} -d

RewriteRule ^ - [L]

RewriteRule ^ index.html [L]

Cette configuration permet aux routes React de fonctionner après un rafraîchissement du navigateur.

VI. Connexion frontend <-> API

1. Configuration de l'URL de l'API

Le frontend utilise la variable d'environnement Vite suivante :

VITE_API_URL=https://api.test.alexandrebourlier.fr/api/v1

Cette variable est intégrée au moment du build React.

L'application React communique donc avec l'API Laravel via :

https://api.test.alexandrebourlier.fr/api/v1

2. Fonctionnement du CORS

Comme le frontend et l'API sont hébergés sur deux sous-domaines différents, une configuration CORS était nécessaire côté Laravel.

Le fichier **config/cors.php** a été configuré afin d'autoriser uniquement l'origine du frontend :

```
'allowed_origins' => [
    'https://test.alexandrebourlier.fr',
],
```

Les environnements locaux ont également été ajoutés pendant les tests :

```
'http://localhost:5173',
'http://localhost:4173',
```

Cela permet d'autoriser :

- Le serveur de développement Vite
- Le mode preview Vite
- Le frontend de préproduction

3. Résolution du CORS en production

Une erreur CORS est apparue lors des premiers tests :

No Access-Control-Allow-Origin header is present

Elle a été corrigée en ajoutant l'origine de production dans **config/cors.php**, puis en vidant le cache Laravel :

```
php artisan optimize:clear
php artisan config:cache
```

Après correction, le frontend React a pu consommer correctement les endpoints Laravel.

VII. Sécurité

Plusieurs mesures ont été mises en place pendant la mise en ligne.

- Le mode debug Laravel a été désactivé
- L'application utilise HTTPS sur le frontend et l'API
- Les URLs générées par Laravel ont été forcées en HTTPS afin d'éviter les formulaires non sécurisés dans OpenAdmin

Dans **AppServiceProvider.php** :

```
if (app()->environment('production')) {
    URL::forceScheme('https');
}
```

Une correction spécifique a également été appliquée au formulaire de connexion OpenAdmin afin que l'action du formulaire utilise une URL HTTPS :

```
<form action="{{ secure_url('admin/auth/login') }}" method="post" autocomplete="on">
  <input type="hidden" name="_token" value="{{ csrf_token() }}">
  <div class="mb-3">
    @if($errors->has('username'))
      <div class="alert alert-danger">{{ $errors->first('username') }}</div>
    @endif
    <label for="username" class="form-label">{{ __('admin.username') }}</label>
    <div class="input-group mb-3">
      <span class="input-group-text"><i class="icon-user"></i></span>
      <input type="text" class="form-control" placeholder="{{ __('admin.username') }}" name="username" id="username" value="{{ old('username') }}" autocomplete="username">
    </div>
  </div>
  <div class="mb-3">
    <label for="password" class="form-label">{{ __('admin.password') }}</label>
    <div class="input-group mb-3">
      <span class="input-group-text"><i class="icon-eye"></i></span>
      <input type="password" class="form-control" placeholder="{{ __('admin.password') }}" name="password" id="password" autocomplete="current-password" required>
    </div>
  </div>
</form>
```

Cela a permis de résoudre l'alerte navigateur indiquant que le formulaire n'était pas sécurisé.

VIII. Tests de validation

1. Tests de l'API

Plusieurs routes API ont été testées directement dans le navigateur.

Exemples :

<https://api.test.alexandrebourlier.fr/api/v1/public/home>

<https://api.test.alexandrebourlier.fr/api/v1/posts>

Les routes ont renvoyé des réponses JSON correctes

Les erreurs rencontrées ont été analysées avec :

Tail -n 100 storage/logs/laravel.log

Et avec **php artisan tinker**

Pour vérifier les valeurs de configuration réellement utilisées par Laravel.

2. Tests frontend

Le frontend a été testé sur :

<https://test.alexandrebourlier.fr>

Les routes React ont été vérifiées :

/contact

/mentions-legales

/conditions-generales-utilisation

/politique-confidentialite

/club

Les rafraîchissements navigateurs fonctionnent grâce au **.htaccess** React Router.

3. Test du formulaire de contact

Le formulaire de contact a été testé en environnement distant.

Les points vérifiés :

- Affichage du formulaire
- Validation frontend
- Appel vers l'API Laravel
- Réponse serveur
- Gestion des messages de succès ou d'erreur

Le fonctionnement HTTPS a également été vérifié afin de garantir que les données ne transitent pas via une URL non sécurisée.

4. Vérifications finales

Les vérifications finales ont porté sur :

- Affichage des pages publiques
- Chargement des images
- Affichage des articles
- Accès aux documents
- Affichage des classements
- Accès à l'administration OpenAdmin
- Fonctionnement des routes API
- Validité du certificat SSL
- Absence d'erreurs CORS
- Fonctionnement de React Router
- Consultation mobile et desktop.

Le site a ensuite été parcouru entièrement afin de vérifier le bon fonctionnement général.

IX. Bilan de la mise en production

1. Difficultés rencontrées

Plusieurs difficultés ont été rencontrées pendant la mise en ligne.

La première concernait le routage React. Après déploiement du build, certaines routes renvoyaient une erreur 404 après rafraîchissement. Le problème a été corrigé avec un **.htaccess** redirigeant les routes inconnues vers **index.html**.

Une seconde difficulté concernait la configuration CORS. Le frontend était bien déployé, mais certaines requêtes vers l'API étaient bloquées car l'origine de production n'était pas autorisée.

Un autre problème important concernait la base de données. Laravel continuait d'utiliser d'anciens identifiants après modification du fichier **.env**. Le cache de configuration et l'ancien fichier **.env.production** ont été identifiés comme cause du problème.

Enfin, une alerte navigateur est apparue sur le formulaire de connexion OpenAdmin. L'action du formulaire était générée en http alors que le site utilisait HTTPS.

2. Solutions apportées

Les solutions suivantes ont été mises en place :

- Séparation claire entre frontend React et backend Laravel
- Configuration des sous-domaines
- Activation des certificats SSL
- Configuration du **.env** Laravel
- Correction du cache de configuration Laravel
- Configuration CORS adaptée
- Ajout du **.htaccess** React Router
- Correction du formulaire OpenAdmin en HTTPS
- Ajout des attributs **autocomplete** sur le formulaire de connexion
- Validation des endpoints API
- Vérification complète du frontend

La mise en ligne a permis de valider l'architecture découplée du projet BCJ37 dans un environnement serveur réel.

Le projet dispose désormais :

- D'un frontend React fonctionnel en HTTPS
- D'une API Laravel accessible en HTTPS
- D'une administration OpenAdmin opérationnelle
- D'une configuration CORS cohérente
- D'un routage React fonctionnel
- D'une base de données connectée
- D'une structure adaptée à une mise en production complète.

Lien Site web (préproduction) : <https://test.alexandrebourlier.fr>

Lien API : <https://api.test.alexandrebourlier.fr>

Lien GitHub du projet : https://github.com/AlexBourlier/Billard_bcj

X. Bugs rencontrés et résolutions

Bug #1 –Erreur de contraintes SQL (foreign key)

Cause identifiée

Nom de contrainte dupliqué lors des migrations

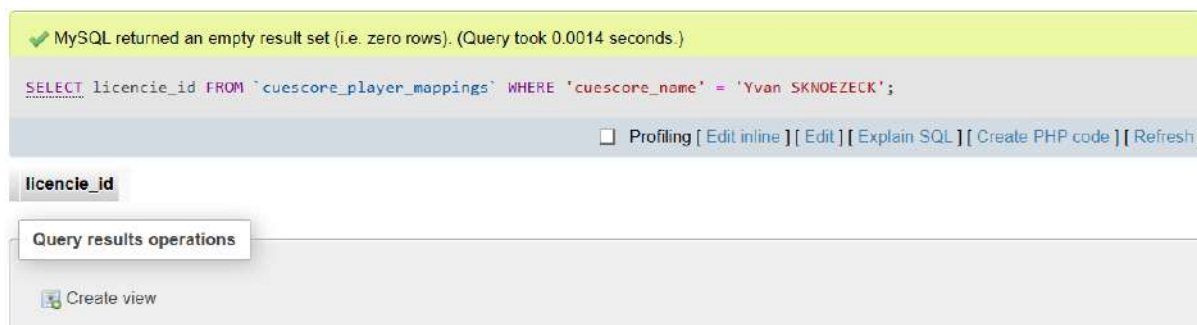
Résolution

- Suppression des index redondants
- Nettoyage des migrations

Bug #2 –Mapping joueur non trouvé.

Causes identifiées

Requête SQL incorrecte :



(Chaîne au lieu de colonne)

Résolution

- Correction de la requête
- Relance du matching



Bug #3 – Matching non exécuté automatiquement

Cause identifiée

Le matching était lancé manuellement après import

Résolution

- Intégration du matching dans la commande **cuescore:import**
- Automatisation complète du pipeline.

Bug #4 – Non-reconnaissance des équipes du club

Cause identifiée

Problème de normalisation (accents : « JOUÉ LÈS TOURS »)

Résolution

- Création d'une fonction de normalisation
- Suppression des accents et uniformisation

16) Conclusion

Le projet BCJ37 a permis de concevoir et de déployer une application web moderne reposant sur une architecture découplée entre le frontend et le backend. Grâce à l'utilisation de React/Vite pour l'interface utilisateur et de Laravel pour l'API REST, la plateforme bénéficie d'une structure claire, maintenable et évolutive. Cette refonte a également permis de moderniser l'expérience utilisateur tout en conservant les bases métier déjà présentes dans l'application initiale.

Ce projet m'a permis de mettre en pratique de nombreuses compétences techniques liées au développement web full stack : création d'API REST, gestion d'une base de données MySQL, développement d'une interface React dynamique, sécurisation HTTPS, configuration CORS, gestion des médias et déploiement sur un serveur distant.

Au-delà des aspects techniques, ce projet m'a surtout permis de mieux comprendre le fonctionnement global d'une architecture web moderne découplée. Les différentes difficultés rencontrées durant le développement ont constitué une part importante de l'apprentissage. Les problèmes liés au routage React en production, à la configuration CORS, au cache Laravel, aux erreurs de migration SQL ou encore à la sécurisation des formulaires ont nécessité des phases de recherche, d'analyse et de résolution proches de situations réelles rencontrées en environnement professionnel.

Le développement du pipeline d'import des licenciés ainsi que l'intégration des données CueScore ont également permis d'aborder des problématiques plus avancées : récupération et transformation de données externes, normalisation, validation métier, historisation et automatisation des traitements. La mise en place d'OpenAdmin et des tâches automatisées via le scheduler Laravel a renforcé l'aspect professionnel du projet en apportant des outils d'administration et de maintenance adaptés aux besoins du club.

Le projet reste néanmoins évolutif et plusieurs améliorations peuvent encore être envisagées. L'interface utilisateur pourrait être davantage travaillée graphiquement afin de se rapprocher plus fidèlement des maquettes initiales. Certaines fonctionnalités pourraient également être enrichies, notamment :

- L'ajout d'un système d'authentification utilisateur ;
- La création de tableaux de bord statistiques ;
- L'amélioration du cache API ;
- La mise en place des tests automatisés plus complets ;
- L'optimisation des performances frontend ;
- Le développement d'une véritable documentation Swagger/OpenAPI.

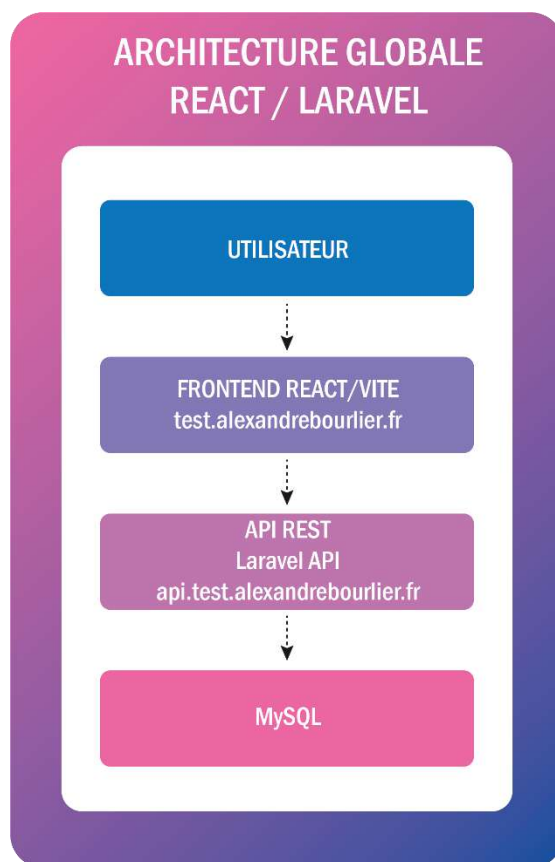
À plus long terme, l'architecture mise en place permettrait également d'envisager :

- Une application mobile ;
- Des API publiques sécurisées ;
- L'ajout de nouvelles disciplines ou compétitions ;
- Des fonctionnalités temps réel ;
- Une administration plus avancée des contenus et des statistiques sportives.

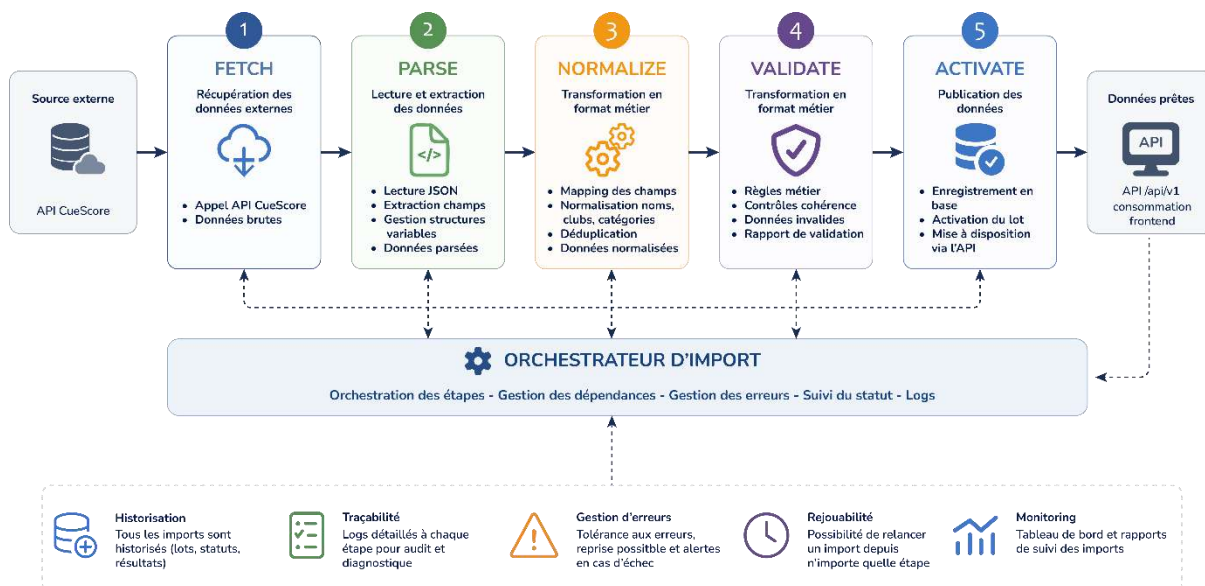
Au terme du projet, la plateforme BCJ37 est pleinement fonctionnelle, sécurisée et exploitable dans un environnement de production. Cette expérience a renforcé mes compétences techniques, ma capacité d'analyse, ma méthodologie de développement ainsi que ma compréhension des enjeux liés à la conception, au déploiement et à la maintenance d'une application web moderne complète.

17) Annexes

I. Annexe 1 – Architecture global React / Laravel



II. Annexe 2 – Pipeline d'import CueScore

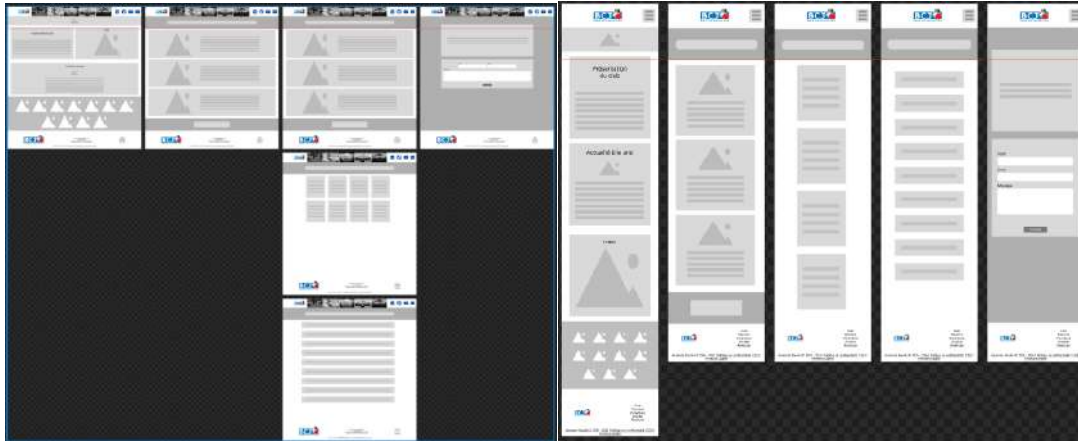


III. Annexe 3 – Zoning – Wireframe et Maquette



Lien Excalidraw zoning :

<https://excalidraw.com/#json=PArZclV83b4KysB9ehNXU,Xd8G2kHaMAoiw22EB9OuEw>



Lien Figma : <https://www.figma.com/design/YYCtOGLyKKVlcQR1AXRebA/bcj?node-id=0-1&t=GxX4cD2KGhSnlqAe-1>

IV. Annexe 4 – Script de création de la base

```
-- =====
-- BCJ37 – Script de création de la base de données
-- Schéma normalisé, cohérent avec le MCD/MLD.

--

-- SGBD : MySQL 8 – Moteur InnoDB – Encodage utf8mb4
-- =====

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

-- -----
-- Suppression préalable (ordre indifférent grâce à la ligne ci-dessus)
-- -----

DROP TABLE IF EXISTS `club_matching_rule`;
DROP TABLE IF EXISTS `cuescore_player_mapping`;
DROP TABLE IF EXISTS `cuescore_ranking_entry`;
```

```

DROP TABLE IF EXISTS `cuescore_ranking_fetch`;
DROP TABLE IF EXISTS `cuescore_ranking`;
DROP TABLE IF EXISTS `licencie`;
DROP TABLE IF EXISTS `calendar_event_link`;
DROP TABLE IF EXISTS `calendar_event`;
DROP TABLE IF EXISTS `calendar`;
DROP TABLE IF EXISTS `document`;
DROP TABLE IF EXISTS `post`;
DROP TABLE IF EXISTS `niveau`;
DROP TABLE IF EXISTS `discipline`;

SET FOREIGN_KEY_CHECKS = 1;

-- =====
-- 1. TABLES DE RÉFÉRENCE
-- =====

-- -----
-- DISCIPLINE : les disciplines du billard (carambole, snooker, ...)
-- -----

CREATE TABLE `discipline` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `nom` VARCHAR(100) NOT NULL,
  `slug` VARCHAR(100) NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_discipline_slug` (`slug`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- -----
-- NIVEAU : périmètre géographique d'une compétition
-- (remplace l'ancienne colonne « scope »)
-- -----

CREATE TABLE `niveau` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `nom` VARCHAR(100) NOT NULL,
  `slug` VARCHAR(100) NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_niveau_slug` (`slug`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 2. CONTENU ÉDITORIAL
-- =====

-- -----
-- POST : articles / actualités du site
-- -----

CREATE TABLE `post` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `title` VARCHAR(150) NOT NULL,
  `slug` VARCHAR(255) NOT NULL,
  `excerpt` VARCHAR(255) DEFAULT NULL,
  `content` TEXT NOT NULL,
  `thumbnail` VARCHAR(255) DEFAULT NULL,
  `discipline_id` INT UNSIGNED DEFAULT NULL,
  `year` SMALLINT NOT NULL,
  `favoris` TINYINT(1) NOT NULL DEFAULT 0,
  `created_at` TIMESTAMP NULL DEFAULT NULL,
  `updated_at` TIMESTAMP NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_post_slug` (`slug`),
  KEY `idx_post_discipline` (`discipline_id`),
  CONSTRAINT `fk_post_discipline`
    FOREIGN KEY (`discipline_id`) REFERENCES `discipline` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

-----
-- DOCUMENT : fichiers téléchargeables (règlements, classements PDF...)
-----
CREATE TABLE `document` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `title` VARCHAR(150) NOT NULL,
  `file` VARCHAR(255) NOT NULL,
  `discipline_id` INT UNSIGNED NOT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_document_discipline` (`discipline_id`),
  CONSTRAINT `fk_document_discipline`
    FOREIGN KEY (`discipline_id`) REFERENCES `discipline` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 3. CALENDRIERS ET ÉVÉNEMENTS
-- =====

-----
-- CALENDAR : un calendrier = une discipline x un niveau
-- (résolution de la relation N:N entre DISCIPLINE et NIVEAU)
-----
CREATE TABLE `calendar` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `discipline_id` INT UNSIGNED NOT NULL,
  `niveau_id` INT UNSIGNED NOT NULL,
  `name` VARCHAR(255) DEFAULT NULL,
  `slug` VARCHAR(255) NOT NULL,
  `source_type` VARCHAR(50) DEFAULT NULL,
  `is_active` TINYINT(1) NOT NULL DEFAULT 1,
  `created_at` TIMESTAMP NULL DEFAULT NULL,
  `updated_at` TIMESTAMP NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_calendar_slug` (`slug`),
  UNIQUE KEY `uk_calendar_discipline_niveau` (`discipline_id`, `niveau_id`),
  KEY `idx_calendar_niveau` (`niveau_id`),
  CONSTRAINT `fk_calendar_discipline`
    FOREIGN KEY (`discipline_id`) REFERENCES `discipline` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_calendar_niveau`
    FOREIGN KEY (`niveau_id`) REFERENCES `niveau` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-----
-- CALENDAR_EVENT : un événement appartient à un calendrier
-----
CREATE TABLE `calendar_event` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `calendar_id` BIGINT UNSIGNED NOT NULL,
  `external_id` VARCHAR(255) DEFAULT NULL,
  `date_debut` DATETIME NOT NULL,
  `date_fin` DATETIME NOT NULL,
  `date_limite` DATETIME DEFAULT NULL,
  `titre` VARCHAR(255) NOT NULL,
  `lieu` VARCHAR(255) NOT NULL,
  `club` VARCHAR(255) DEFAULT NULL,
  `url` TEXT DEFAULT NULL,
  `status` VARCHAR(100) DEFAULT NULL,
  `created_at` TIMESTAMP NULL DEFAULT NULL,
  `updated_at` TIMESTAMP NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_event_calendar` (`calendar_id`),
  CONSTRAINT `fk_event_calendar`

```

```

    FOREIGN KEY (`calendar_id`) REFERENCES `calendar` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-----

-- CALENDAR_EVENT_LINK : liens externes rattachés à un événement
-- (un tournoi peut exposer plusieurs liens : mixte, féminin, U18...)
-----

CREATE TABLE `calendar_event_link` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `calendar_event_id` BIGINT UNSIGNED NOT NULL,
  `category` VARCHAR(100) NOT NULL,
  `label` VARCHAR(255) DEFAULT NULL,
  `url` TEXT NOT NULL,
  `sort_order` INT DEFAULT NULL,
  `created_at` TIMESTAMP NULL DEFAULT NULL,
  `updated_at` TIMESTAMP NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_link_event` (`calendar_event_id`),
  CONSTRAINT `fk_link_event`
    FOREIGN KEY (`calendar_event_id`) REFERENCES `calendar_event` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-----

-- =====
-- 4. LICENCIÉS ET CLASSEMENTS CUESCORE
-- =====
-----

-- LICENCIE : licencié de la fédération
-----

CREATE TABLE `licencie` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `licence` VARCHAR(255) NOT NULL,
  `nom` VARCHAR(255) NOT NULL,
  `prenom` VARCHAR(255) NOT NULL,
  `url` VARCHAR(255) DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_licencie_licence` (`licence`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-----

-- CUESCORE_RANKING : un classement importé depuis Cuescore,
-- rattaché à une discipline et un niveau
-----

CREATE TABLE `cuescore_ranking` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `discipline_id` INT UNSIGNED NOT NULL,
  `niveau_id` INT UNSIGNED NOT NULL,
  `cuescore_id` VARCHAR(255) NOT NULL,
  `name` VARCHAR(255) NOT NULL,
  `ranking_type` VARCHAR(30) NOT NULL,
  `season` VARCHAR(20) NOT NULL,
  `is_active` TINYINT(1) NOT NULL DEFAULT 0,
  `sort_order` INT NOT NULL DEFAULT 0,
  `created_at` TIMESTAMP NULL DEFAULT NULL,
  `updated_at` TIMESTAMP NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_ranking_cuescore_id` (`cuescore_id`),
  KEY `idx_ranking_discipline` (`discipline_id`),
  KEY `idx_ranking_niveau` (`niveau_id`),
  CONSTRAINT `fk_ranking_discipline`
    FOREIGN KEY (`discipline_id`) REFERENCES `discipline` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_ranking_niveau`
    FOREIGN KEY (`niveau_id`) REFERENCES `niveau` (`id`)

```



```

    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-----
-- CUESCORE_RANKING_FETCH : historise chaque récupération (appel) d'un
-- classement depuis l'API Cuescore – traçabilité et diagnostic
-----
CREATE TABLE `cuescore_ranking_fetch` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `cuescore_ranking_id` BIGINT UNSIGNED NOT NULL,
  `status` VARCHAR(30) NOT NULL,
  `fetched_at` TIMESTAMP NULL DEFAULT NULL,
  `http_status` SMALLINT UNSIGNED DEFAULT NULL,
  `payload_hash` VARCHAR(64) DEFAULT NULL,
  `records_count` INT UNSIGNED DEFAULT NULL,
  `error_code` VARCHAR(50) DEFAULT NULL,
  `error_message` TEXT DEFAULT NULL,
  `created_at` TIMESTAMP NULL DEFAULT NULL,
  `updated_at` TIMESTAMP NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_fetch_ranking` (`cuescore_ranking_id`),
  CONSTRAINT `fk_fetch_ranking`
    FOREIGN KEY (`cuescore_ranking_id`) REFERENCES `cuescore_ranking` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-----
-- CUESCORE_RANKING_ENTRY : une ligne (un joueur) dans un classement,
-- rattachée au classement et à la récupération qui l'a produite
-----
CREATE TABLE `cuescore_ranking_entry` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `cuescore_ranking_id` BIGINT UNSIGNED NOT NULL,
  `cuescore_ranking_fetch_id` BIGINT UNSIGNED NOT NULL,
  `rank_position` INT UNSIGNED DEFAULT NULL,
  `participant_name` VARCHAR(255) DEFAULT NULL,
  `points` DECIMAL(10,2) DEFAULT NULL,
  `played` INT UNSIGNED DEFAULT NULL,
  `wins` INT UNSIGNED DEFAULT NULL,
  `losses` INT UNSIGNED DEFAULT NULL,
  `created_at` TIMESTAMP NULL DEFAULT NULL,
  `updated_at` TIMESTAMP NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_entry_ranking` (`cuescore_ranking_id`),
  KEY `idx_entry_fetch` (`cuescore_ranking_fetch_id`),
  CONSTRAINT `fk_entry_ranking`
    FOREIGN KEY (`cuescore_ranking_id`) REFERENCES `cuescore_ranking` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_entry_fetch`
    FOREIGN KEY (`cuescore_ranking_fetch_id`) REFERENCES `cuescore_ranking_fetch` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-----
-- CUESCORE_PLAYER_MAPPING : table de correspondance entre un
-- participant Cuescore et un licencié de la fédération
-----
CREATE TABLE `cuescore_player_mapping` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `licencie_id` BIGINT UNSIGNED DEFAULT NULL,
  `cuescore_participant_id` VARCHAR(255) NOT NULL,
  `cuescore_name` VARCHAR(255) NOT NULL,
  `matching_method` VARCHAR(30) NOT NULL,
  `confidence_score` TINYINT UNSIGNED DEFAULT NULL,
  `is_confirmed` TINYINT(1) NOT NULL DEFAULT 0,
  `created_at` TIMESTAMP NULL DEFAULT NULL,

```



```

`updated_at`          TIMESTAMP          NULL DEFAULT NULL,
PRIMARY KEY (`id`),
UNIQUE KEY `uk_mapping_participant` (`cuescore_participant_id`),
KEY `idx_mapping_licencie` (`licencie_id`),
CONSTRAINT `fk_mapping_licencie`
  FOREIGN KEY (`licencie_id`) REFERENCES `licencie` (`id`)
  ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-----
-- CLUB_MATCHING_RULE : règles de reconnaissance du club dans les
-- données Cuescore (le nom du club peut apparaître sous des variantes)
-----
CREATE TABLE `club_matching_rule` (
  `id`                INT UNSIGNED      NOT NULL AUTO_INCREMENT,
  `club_reference_name` VARCHAR(255)    NOT NULL,
  `matching_mode`     VARCHAR(30)       NOT NULL, -- contains / equals / starts_with
  `matching_value`    VARCHAR(255)     NOT NULL,
  `is_active`         TINYINT(1)        NOT NULL DEFAULT 1,
  `created_at`        TIMESTAMP         NULL DEFAULT NULL,
  `updated_at`        TIMESTAMP         NULL DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 5. DONNÉES DE RÉFÉRENCE (jeu d'amorçage)
-- =====

INSERT INTO `discipline` (`id`, `nom`, `slug`) VALUES
(1, 'Blackball', 'blackball'),
(2, 'Carambole', 'carambole'),
(3, 'Snooker', 'snooker'),
(4, 'Américain', 'americain');

INSERT INTO `niveau` (`id`, `nom`, `slug`) VALUES
(1, 'International', 'international'),
(2, 'National', 'national'),
(3, 'Régional', 'regional'),
(4, 'Départemental', 'departemental');

```

Les sources suivantes ont été utilisées tout au long du développement du projet BCJ37 afin de concevoir, développer, sécuriser et déployer l'application web

V. Annexe 5 – Code composants métier

1. Orchestrateur du pipeline d'import des licenciés

Composant métier le plus représentatif : il **orchestre** les étapes de l'import (récupération -> analyse -> normalisation -> snapshot -> validations -> activation -> projection) dans un **style défensif**. Chaque typed'erreur est capturé, journalisé puis relancé, aucun import invalide n'est publié. La classe est *final*, ses dépendances sont **injectées** et **en lecture seule** (*readOnly*), et le fichier active le typage strict (*declare(strict_types=1)*)

```

<?php
declare(strict_types=1);
namespace App\Domain\LicenseImport\Pipeline;
// ... imports du domaine LicenseImport (Fetch, Parsing, Normalization, Validation,
Activation...)
final class TelematLicenseImportOrchestrator
{

```

```

public function __construct(
    private readonly TelematFetchConfigFactory $fetchConfigFactory,
    private readonly TelematFetcherInterface $httpClient,
    private readonly TelematBatchLifecycleManager $batchLifecycleManager,
    private readonly TelematHtmlTableParser $htmlTableParser,
    private readonly TelematRowNormalizer $rowNormalizer,
    private readonly TelematSnapshotPersister $snapshotPersister,
    private readonly TelematMinimalBatchValidator $minimalBatchValidator,
    private readonly TelematComparativeBatchValidator $comparativeBatchValidator,
    private readonly TelematBatchActivator $batchActivator,
    private readonly TelematFailureManager $failureManager,
    private readonly TelematPipelineLogger $logger,
    private readonly TelematBatchReportManager $batchReportManager,
    private readonly TelematLicenseProjector $licenseProjector,
    private readonly TelematProjectionEligibilityDecider $projectionEligibilityDecider,
) {
}
public function run(LicenseImportExecutionContext $context): LicenseImportBatch
{
    $batch = $this->batchLifecycleManager->start($context);
    try {
        $fetchResult = $this->dispatchFetch($context);
        $this->batchLifecycleManager->transition($batch, BatchStatus::Fetched);
        $parsedTable = $this->htmlTableParser->parse($fetchResult->html, $context);
        $this->batchLifecycleManager->transition($batch, BatchStatus::Parsed);
        $normalizedRows = $this->rowNormalizer->normalize($parsedTable, $context);
        $this->snapshotPersister->persist(
            batch: $batch,
            parsedHeaders: $parsedTable['headers'],
            rows: $normalizedRows,
            context: $context,
        );
        // Deux niveaux de validation avant toute publication (garde-fous).
        $this->minimalBatchValidator->validate($batch, $context);
        $this->batchLifecycleManager->transition($batch,
BatchStatus::ValidatedMinimal);
        $this->comparativeBatchValidator->validate($batch, $context);
        $this->batchLifecycleManager->transition($batch,
BatchStatus::ValidatedComparative);
        $this->batchActivator->activate($batch, $context);
        $batch->refresh();
        $projectionResult = $this->resolveProjectionResult($batch, $context);
        $this->appendProjectionSummary($batch, $projectionResult);
        $this->batchLifecycleManager->refreshIssueCounters($batch);
        $this->batchReportManager->writeSafely($batch);
        return $batch;
    } catch (TelematFetchException $exception) {
        $this->failureManager->handleFetchFailure($batch, $context, $exception);
        throw $exception;
    } catch (TelematHtmlParseException $exception) {
        $this->failureManager->handleStageFailure($batch, $context, $exception,
            'parse', 'parse_failed');
        throw $exception;
    } catch (MinimalValidationFailedException $exception) {
        $this->failureManager->handleStageFailure($batch, $context, $exception,
            'minimal_validation', 'minimal_validation_failed');
        throw $exception;
    } catch (ComparativeValidationFailedException $exception) {
        $this->failureManager->handleStageFailure($batch, $context, $exception,
            'comparative_validation', 'comparative_validation_failed');
        throw $exception;
    } catch (BatchActivationException $exception) {
        $this->failureManager->handleStageFailure($batch, $context, $exception,
            'activation', 'activation_failed');
        throw $exception;
    } catch (Throwable $exception) {

```

```

        // Toute erreur inattendue est tracée : aucune défaillance silencieuse.
        $this->failureManager->handleUnexpectedFailure($batch, $context, $exception);
        throw $exception;
    }
}
private function dispatchFetch(LicenseImportExecutionContext $context):
TelematFetchResult

    $config = $this->fetchConfigFactory->make();
    $this->logger->info('license_import.pipeline.fetch.dispatching', [
        'source' => $context->source,
        'trigger_type' => $context->triggerType->value,
        'dry_run' => $context->dryRun,
        'url' => $config->url,
    ]);
    return $this->httpFetcher->fetch($config);
}
private function resolveProjectionResult(
    LicenseImportBatch $batch,
    LicenseImportExecutionContext $context,
): ProjectionResult {
    $decision = $this->projectionEligibilityDecider->decide(
        $batch, $context, $this->licenseProjector->strategyName(),
    );
    if ($decision->shouldSkip()) {
        return ProjectionResult::skipped($decision->reason ?? 'projection_skipped');
    }
    if ($decision->isPreview()) {
        return $this->licenseProjector->preview($batch);
    }
    return $this->licenseProjector->project($batch);
}
// ... appendProjectionSummary() et logCompletion() : méthodes de journalisation
(omises ici)
}

```

2. Composant métier de correspondance discipline / identifiant

Composant métier **pur et testable** qui centralise la correspondance entre le slug d'une discipline et son identifiant. Il fournit un **point unique de validation** (*isValidSlug*) et évite la dispersion de cette logique dans le code (principe DRY)

```

<?php
namespace App\Support;
class DisciplineMapper
{
    public const DISCIPLINES = [
        'blackball' => 1,
        'carambole' => 2,
        'snooker' => 3,
        'americain' => 4,
    ];
    public static function idFromSlug(string $slug): ?int
    {
        return self::DISCIPLINES[$slug] ?? null; // null si slug inconnu : pas d'exception
        non maîtrisée
    }
    public static function slugFromId(int|string|null $id): ?string
    {
        if ($id === null) {
            return null;
        }
        $id = (int) $id;
        return array_flip(self::DISCIPLINES)[$id] ?? null;
    }
}

```

```

public static function isValidSlug(string $slug): bool
{
    return array_key_exists($slug, self::DISCIPLINES);
}
public static function slugs(): array
{
}
}

```

VI. Annexe 6 – Code des composants d'accès aux données

1. Modèle Eloquent Calendar (relations, filtres réutilisables, sécurité)

Composant d'accès aux données via l'ORM **Eloquent** (requêtes **paramétrées** -> protection native contre l'injection SQL). Le modèle centralise les **relations**, des **filtres réutilisables** (query scopes), les **valeurs de référence validées** (constantes + isValidDiscipline / isValidScope) et protège contre le mass-assignment via la liste **\$fillable**

```

<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;
class Calendar extends Model
{
    protected $table = 'calendars';
    // Liste blanche des champs assignables en masse (anti mass-assignment).
    protected $fillable = ['discipline', 'scope', 'name', 'slug', 'source_type',
    'is_active'];
    // Typage automatique exposé à l'API.
    protected $casts = ['is_active' => 'boolean'];
    protected $appends = ['display_name'];
    // Valeurs de référence (garantissent l'intégrité au niveau applicatif).
    public const DISCIPLINE_CARAMBOLE = 'carambole';
    public const DISCIPLINE_SNOOKER = 'snooker';
    public const DISCIPLINE_AMERICAIN = 'américain';
    public const DISCIPLINE_BLACKBALL = 'blackball';
    public const SCOPE_INTERNATIONAL = 'international';
    public const SCOPE_NATIONAL = 'national';
    public const SCOPE_REGIONAL = 'regional';
    public const SCOPE_DEPARTEMENTAL = 'departemental';
    public static function disciplines(): array
    {
        return [self::DISCIPLINE_CARAMBOLE, self::DISCIPLINE_SNOOKER,
            self::DISCIPLINE_AMERICAIN, self::DISCIPLINE_BLACKBALL];
    }
    public static function scopes(): array
    {
        return [self::SCOPE_INTERNATIONAL, self::SCOPE_NATIONAL,
            self::SCOPE_REGIONAL, self::SCOPE_DEPARTEMENTAL];
    }
    public static function isValidDiscipline(string $discipline): bool
    {
        return in_array($discipline, self::disciplines(), true);
    }
    public static function isValidScope(string $scope): bool
    {
        return in_array($scope, self::scopes(), true);
    }
    // --- Relations ---
    public function events(): HasMany
    {

```

```

        return $this->hasMany(CalendarEvent::class, 'calendar_id')
            ->orderBy('date_debut', 'asc');
    }
    public function upcomingEvents(): HasMany
    {
        return $this->hasMany(CalendarEvent::class, 'calendar_id')
            ->where('date_fin', '>=', now())
            ->orderBy('date_debut');
    }
    // --- Filtres réutilisables (query scopes) : évitent la duplication de requêtes ---
    public function scopeActive(Builder $query): Builder
    {
        return $query->where('is_active', true);
    }
    public function scopeByDiscipline(Builder $query, string $discipline): Builder
    {
        return $query->where('discipline', $discipline);
    }
    public function scopeByScope(Builder $query, string $scope): Builder
    {
        return $query->where('scope', $scope);
    }
    public function scopeOrdered(Builder $query): Builder
    {
        return $query->orderBy('discipline')->orderBy('scope');
    }
    // Accessor : nom d'affichage calculé si aucun nom n'est défini.
    public function getDisplayNameAttribute(): string
    {
        if (!empty($this->name)) {
            return $this->name;
        }
        return ucfirst($this->discipline) . ' - ' . ucfirst($this->scope);
    }
}

```

2. Contrôleur d'API CalendarController (validation + réponse standardisée)

Composant d'accès aux données en **consultation**. Il **valide systématiquement** les paramètres avant toute requête (*abort_unless* -> 404 si invalide), optimise l'accès (*withCount* évite le problème **N+1**), contrôle les chmaps exposés via une **Resource** (pas de fuite de colonnes internes) et renvoie le format homogène *data/meta/links/error*.

```

<?php
namespace App\Http\Controllers\Api;
use App\Http\Controllers\Controller;
use App\Http\Resources\CalendarEventResource;
use App\Http\Resources\CalendarResource;
use App\Models\Calendar;
use Illuminate\Http\JsonResponse;
class CalendarController extends Controller
{
    // Liste des calendriers actifs.
    public function index(): JsonResponse
    {
        $calendars = Calendar::query()->active()->ordered()->get();
        return response()->json([
            'data' => CalendarResource::collection($calendars),
            'meta' => ['count' => $calendars->count()],
            'links' => [],
            'error' => null,
        ]);
    }
    // Calendriers actifs d'une discipline donnée.
    public function byDiscipline(string $discipline): JsonResponse
    {

```

```

{
    // On rejette une discipline inconnue AVANT d'interroger la base.
    $this->abortIfInvalidDiscipline($discipline);
    $calendars = Calendar::query()
        ->active()
        ->byDiscipline($discipline)
        ->ordered()
        ->withCount('events') // compte les événements en une requête (anti N+1)

    return response()->json([
        'data' => CalendarResource::collection($calendars),
        'meta' => ['discipline' => $discipline, 'count' => $calendars->count()],
        'links' => [],
        'error' => null,
    ]);
}
// Un calendrier précis (discipline + scope) avec ses événements et leurs liens.
public function byDisciplineAndScope(string $discipline, string $scope): JsonResponse
{
    $this->abortIfInvalidDiscipline($discipline);
    $this->abortIfInvalidScope($scope);
    $calendar = Calendar::query()
        ->active()
        ->byDiscipline($discipline)
        ->byScope($scope)
        ->with(['events.links', 'events.calendar'])
        ->firstOrFail();
    return response()->json([
        'data' => [
            'calendar' => new CalendarResource($calendar),
            'events' => CalendarEventResource::collection($calendar->events),
        ],
        'meta' => ['discipline' => $discipline, 'scope' => $scope,
            'count' => $calendar->events->count()],
        'links' => [],
        'error' => null,
    ]);
}
// --- Validation des paramètres d'entrée (style défensif) ---
private function abortIfInvalidDiscipline(string $discipline): void
{
    abort_unless(Calendar::isValidDiscipline($discipline), 404, 'Discipline
invalide.');
```

VII. Documentation Laravel

Documentation officielle Laravel : <https://laravel.com/docs/10.x>

Documentation principale utilisée pour :

- Création de l'API REST (<https://laravel.com/docs/10.x/eloquent-resources>)
- Configuration des routes (<https://laravel.com/docs/10.x/routing>)
- Gestion du stockage (<https://laravel.com/docs/10.x/filesystem>)
- Configuration serveur (<https://laravel.com/docs/10.x/deployment>)
- Sécurité et middleware (<https://laravel.com/docs/10.x/middleware>)

Laravel Routing Documentation : <https://laravel.com/docs/10.x/routing>

Utilisée pour :

- Création des routes API
- Gestion des endpoints REST
- Configuration CORS

Laravel Configuration Documentation : <https://laravel.com/docs/10.x/configuration>

Utilisée pour :

- Configuration du fichier .env
- Gestion du cache Laravel
- Configuration serveur

Laravel Vite Documentation : <https://laravel.com/docs/10.x/vite>

Utilisée pour :

- Intégration du frontend Vite
- Gestion des assets frontend

Laravel CSRF Protection Documentation : <https://laravel.com/docs/10.x/csrf>

Utilisée pour :

- Sécurisation des formulaires
- Gestion des protections CSRF

VIII. Documentation React et Vite

Documentation officielle React : <https://react.dev/reference/react>

Documentation utilisée pour :

- Développement des composants ;
- Gestion des états ;
- Architecture frontend SPA.

Documentation officielle Vite <https://vite.dev/guide/>

Utilisée pour :

- Configuration du frontend ;
- Build de production ;
- Variables d'environnement.

Documentation React Router : <https://reactrouter.com/home>

Utilisée pour :

- Routage SPA ;
- Navigation frontend ;
- Gestion des routes React.

IX. Documentation OpenAdmin

OpenAdmin Documentation : <https://open-admin.org/docs>

Utilisée pour :

- Mise en place du panneau d'administration ;
- Gestion des contenus ;
- Configuration des formulaires d'administration.

X. Documentation MySQL

Documentation officielle MySQL : <https://dev.mysql.com/doc/>

Utilisée pour :

- Configuration de la base de données ;
- Gestion des tables ;
- Requêtes SQL.

XI. Hébergement et déploiement

Documentation o2switch : <https://faq.o2switch.fr/cpanel/>

Utilisée pour :

- Configuration de l'hébergement ;
- Création des sous-domaines ;
- Configuration Apache ;
- Tâches cron.

Let's Encrypt : <https://faq.o2switch.fr/cpanel/securite/lets-encrypt-ssl-gratuit/>

Utilisé pour :

- Installation des certificats SSL ;
- Sécurisation HTTPS.

XII. Ressources complémentaires

Stack Overflow

Utilisé pour :

- Résolution d'erreurs techniques ;
- Problèmes React Router ;
- Erreurs Laravel ;
- Configuration serveur Apache.

GitHub

Utilisé pour :

- Consultation de projets open source ;
- Documentation communautaire ;
- Exemples d'implémentation.